Introduction to Analytics

This lesson introduces analytics, its core functions, and why it is important.

WE'LL COVER THE FOLLOWING

- What is analytics?
- The analytics process
- Importance of analytics

What is analytics?

Analytics, at its core, is the process of taking raw data and extracting actionable insights.

For example, a raw data set would be the historical stock market prices and financials for all of the companies in the S&P 500. By itself, that raw data isn't that useful. With analytical skills, though, you can start to extract understanding from your data. Perhaps, you wish to discover the average market return by year or identify any stocks with outlier performances. Analytics is the process by which you were to understand your data. Moreover, that understanding will allow you to make data-driven decisions.

The analytics process

I like to break down the analytics process into **four** distinct steps:

- Processing the data
- Describing the data
- Cleaning the data
- Visualizing the data

Typically, when starting a new analytics project, you are given a bunch of data and a question you want to answer. First, you need to read those data into a

system for **processing.** Once loaded, you will typically start to calculate various statistics and counts to **describe** the data. This process tends to reveal potential issues and will require you to **clean** the data. At that point, you will likely go back and do a deeper dive into describing and understanding your data. To augment this process, you will then start to **visualize** the data.

Importance of analytics

The goal of this entire process is to start to peel back the layers of your data to try and answer your question or questions of interest. That is the power of analytics. It allows you to start with no knowledge and a bunch of data to data-driven understanding.

This course will introduce you to the key concepts of each step in the processes using the Python programming language. By the end of the course, you will have the ability to extract knowledge and answers from data.

Analytics in Python

This lesson will describe why we are using Python for this course.

WE'LL COVER THE FOLLOWING

- Why Python for analytics?

Why Python for analytics?

Python has quickly become the language of choice when it comes to data analytics, machine learning, and data science.

Other languages provide value in this space as well, such as R and Julia. That being said, in my opinion, Python is the clear leader.

In fact, in May 2019, R fell off Tiobe index's list of the 20 most popular languages, having spent three years in the top tier while Python sat at number 4 with the most significant growth of any language in the top 20.

Python has been so popular because of its applicability to many fields and purposes, its ease of use, and its amazing ecosystem of libraries that support data analytics and machine learning.

We will be using Python exclusively in this course and will discuss many of its great libraries, such as `Numpy`, `Scipy`, `Pandas`, and `Seaborn`.

Getting Started

This lesson will walk you through some basic commands and operations in Python.

WE'LL COVER THE FOLLOWING

- Native Python for science
  - Importing functionalities
  - Lists in Python
  - Functions in Python

Native Python for science

Let's see what Python looks like! I hope you know the basics of Python, but this lesson will hopefully be a good, quick refresher. The first thing you will notice is that Python makes use of whitespace and does not use a use semicolon(;) like some other languages. Here is a very simple example:

```
x = 5
y = 10
print(x+y)
```

Importing functionalities

We will make use of many libraries, some that are pre-installed with Python and some we will have to install ourselves. To get a library use an import statement:

```
from collections import Counter
```

This command imports the class `Counter` from the `collections` library.

`Counter` is a very useful tool for data scientists; it can count the number of times items appear in collections such as lists. For example, in the code below

we will create a list of marriage ages. Using `Counter` we can quickly count the number of times each unique age appears.

Lists in Python

Lists are a useful data structure to store data. They will be studied in more detail during the next lesson. For example:

```
from collections import Counter
marriage_ages = [22, 22, 25, 25, 30, 24, 26, 24, 35]  # create a list
value_counts = Counter(marriage_ages)  # apply the counter functionality
print(value_counts.most_common())
```

You can see that we created a list containing marriage ages using the **[]** at **line 2**. We then fed that list into the `Counter` function at **line 4** to print out the most common values as a list of tuples, at **line 5**.

A **tuple** is a collection inside the (). These tuples contain *two* elements: the value and then the number of times that value appeared in your list. The frequency orders the list of tuples. The value with the most occurrence appears first.

Functions in Python

Functions are also useful. Functions in Python start with the keyword `def` and the *function name* followed by the *inputs* the function expects within brackets. Here is a function that takes in 2 inputs, `x` and `y`, and returns the `sum`:

```
def add_two_numbers(x, y):  # function header
    """
    Takes in two numbers and returns the sum
    parameters
        x : str
            first number
        y : str
            second number
    returns
        x+y
    """
```

```
    z = x + y
    return z  # function return
print(add_two_numbers(100,5))  # function  call
```

Functions can also be *anonymous*, meaning that you don't have to declare them with the above structure. Instead, you can use the `lambda` keyword. Here is the same function as above, but as an anonymous function:

```
y = lambda x, y: x + y  # an anonymous function which takes x and y and input and returns x+y
print(y(100,5))  # call the function
```

Now that you have seen the basics of Python, in the next section we will introduce the most common data structures you will need to know for data processing.

Data Structures

This lesson discusses the common built-in data structures of Python used for data processing.

WE'LL COVER THE FOLLOWING

- Lists in Python
- Dictionaries in Python
- Sets in Python

Lists in Python

As mentioned in the previous section, **lists** are a useful data structure in Python. Lists are *ordered*, *changeable*, and allow for *duplicates*. You create a list with the [].

We can slice a list. *Slicing* means to extract a part of the list. When slicing, the first number is included in the return set while the last number is not. For example:

```
p1 = [1,2,3,4,5,6,7,8,9,10]
p2 = p1[0:5]
```

The list `p2` will be `[1,2,3,4,5]`. Because we'll start from **0**th index of `p1` and will stop at **5**th index slicing all the elements till **4**th index.

> **Note:** You can also write `p2 = p1[0:5]` as `p2 = p1[:5]`.

Here are many of the ways you can use lists.

Please take the time to review the comments and the `print` statements to understand what each line is doing. You can tie the print statements to the code by the lines printed above the results in the output.

```
# This creates the list
depths = [1, 5, 3, 6, 4, 7, 10, 12]

# This outputs the first 5 elements. No number before the : implies 0
first_5_depths = depths[:5]

print("---0---")
print(first_5_depths)

# You can easily sum
print("---1---")
print(sum(depths))

# And take the max
print("---2---")
print(max(depths))

# Slicing with a negative starts from the end, so this returns the last element
print("---3---")
print(depths[-1])

# This returns the end of the list starting from the second to the end
# Nothing after the : implies the end of the list
print("---4---")
print(depths[-2:])

# This returns the second, third, and forth elements
# Remember counting starts at zero!
print("---5---")
print(depths[2:5])

# These commands check if a value is contained in the list
print("---6---")
print(22 in depths)
print(1 in depths)

# This is how you add another value to the end of your list
depths.append(44)
print("---7---")
print(depths)

# You can extend a list with another list
depths.extend([100, 200])
print("---8---")
print(depths)

# You can also modify a value
# This replaces the 4th value with 100
depths[4] = 100
print("---9---")
print(depths)

# Or you can do insert to accomplish the same thing
depths.insert(5, 1000)
print("---10---")
print(depths)
```

When talking about lists, it is useful to discuss **generators**. Generators are objects you can loop over like a list, but they are lazy. It means they don't have to store the entire list in memory. Instead they return the next value in the list only when asked, making them very memory efficient. Because of this property, you can have a function that generates an infinite list; it never stores the value in memory but just keeps track of the last value returned and returns that value + 1 when asked for another value.

Dictionaries in Python

Another useful data type is **dictionary**. Dictionaries hold *key-value* pairs. In other words, there are *unique* **keys** in a dictionary, and each key is associated with a **value**.

The key-value pairs are *unordered* and *changeable*. It means that key-value pairs are stored in an unordered manner because data is accessed using a key rather than index. And we can always change the value of a key when needed.

They are initialized with {}. Let's take a closer look at some functionalities in code!

```
# Initialize the dictionary.
# Keys are first then a : then the value
my_dict = {"age": 22, "birth_year": 1999, "name": "jack", "siblings": ["jill", "jen"]}

# Get the value for the key age
print("---0---")
print(my_dict['age'])

# Check is age is a key
print("---1---")
print('age' in my_dict)

# Check is company is a key
print("---2---")
print('company' in my_dict)

# Get the value for they key age
print("---3---")
print(my_dict.get('age'))

# Get the value for they key company
# If it doesn't exsist, return 1
print("---4---")
print(my_dict.get('company', 1))
```

```
# Return all the keys
print("---5---")
print(my_dict.keys())


# Return all the values
print("---6---")
print(my_dict.values())

# Return all the key, value pairs
print("---7---")
print(my_dict.items())
```

You can also use `defaultdict`, which is a more advanced dictionary data type that allows you to specify a default data type. Let's look at an example:

```
from collections import defaultdict  # import defaultdict class
my_default_dict = defaultdict(int)   # make a default dictionary
my_default_dict['age'] = 22          # adding a key-value pair
print(my_default_dict['company'])    # printing the value of the key "company"
```

In this example, we initialize `defaultdict` with *int*, at **line 1**. That means that it assumes all the values will be integers and that 0 is the default value for any key not assigned. As seen above, when we try and get the value for the `company` at **line 6**, which has yet to be assigned any value in our dictionary, it returns 0. This can be very useful when you want to assume a default value and don't want to use the `get()` functionality as shown previously.

Sets in Python

Sets are another useful data type. Sets are an *unordered* collection of *unique* elements, which means any duplicates are automatically removed. Sets allow you to do operations like *union*, *intersection*, and *difference*. Here's an example:

```
my_set = set()
my_set.add(1)
my_set.add(2)
my_set.add(1)
# Note that the set only contains a single 1 value
print("---0---")
print(my_set)
```

```
my_set2 = set()
my_set2.add(1)
my_set2.add(2)
my_set2.add(3)
my_set2.add(4)
print("---1---")
print(my_set2)

# Prints the overlap
print("---1---")
print(my_set.intersection(my_set2))
print("---2---")

# Prints the combination
print(my_set.union(my_set2))

# Prints the difference (those in my_set but not my_set2)
print("---3---")
print(my_set.difference(my_set2))
```

In the next section, we will look at controlling the flow of your code and learn some helpful functions.

Control Flow and Built-in Functions

This lesson covers how the flow of execution transfers between lines of code depending on the scenarios. It also discusses some built-in functions for data structures studied in the previous lesson.

WE'LL COVER THE FOLLOWING

- Control structures in Python
  - The if-else construct
  - The for construct
- Built-in functions in Python
  - The `sort` function
  - The `zip` function
    - Combining two lists into a tuple
    - Breaking a tuple into two lists

Control structures in Python

The if-else construct

If-then statements are a staple of any programming language. Basically, if you meet a certain condition, then something happens. In Python, `elif` stands for `else if`, meaning that if the previous conditions were not met, check that condition. `Else` is a catch-all condition for any remaining flows. Python follows the following syntax:

```
if condition:
  statements
elif condition:
  statements
else:
  statements
```

Following is a code example:

```
def age_check(age):

    if age > 40:  # if age greater than 40, print "Older than 40"
        print("Older than 40")
    elif age > 30 and age <= 40: # if age greater than 30 and less than or equal to 40, print
        print("Between 30 and 40")
    else:  # if neither of the previous conditions are met, print "Other"
        print("Other")

print(age_check(41))
```

The for construct

Loops allow you to iterate over an iterable. That's not a very helpful definition, so let's consider the most common use case, lists. A loop allows you to iterate over a list or other data types that also allow iteration.

You can contain your iterable in the `enumerate()` command to add a counter to your loop. This is useful if you want to loop over a list of values while still having access to the iterable index.

```
names = ['tyler', 'karen', 'jill']   # list containing names

for i, name in enumerate(names):     # iterating over names
    print("Index: {0}".format(i))    # printing index number
    print("Value: {0}".format(name)) # print the value at the index
```

Speaking of lists and loops, there is an easy way to combine the two in Python called **list comprehensions**. Basically, list comprehensions allow you to create a new list based on an existing list in a condensed way.

Here is a way to create a list of the numbers greater than 5 (up to 14) with a list comprehension.

> **Note:** The range() creates a list of values starting from the first number and ending at the number right before the last number.

So `range(1,6)` creates [1, 2, 3, 4, 5].

We can add an if statement to our list comprehension by adding the condition at the end. Look at the following implementation:

```
numbers_gt_5 = [x for x in range(1,15) if x > 5]  # loop over the range and only keep the val
print(numbers_gt_5)
```

The above program is looping over a range:

```
x for x in range(1,15)
```

It means the loop will iterate 14 times. And in each iteration, an if statement executes. The if statement will be true when `x` is greater than **5**. So the list `numbers_gt_5` will start getting populated once the iterator `x` exceed value of **5**.

We could also increment every value in a list by 1 as below:

```
nums_plus_one = [x + 1 for x in range(5)]
print(nums_plus_one)
```

Built-in functions in Python

The `sort` function

Sorting comes up a lot when talking about lists and is, fortunately, very easy:

```
my_list = [2, 10, 1, -5, 22]
my_list.sort()  # sorting the list

print(my_list)
```

You'll notice it sorts from smallest to largest. If you want more functionality to

You'll notice it sorts from smallest to largest. If you want more functionality to your sorting, I would suggest using the `sorted()` function.

This allows you to reverse the order with the `reverse` parameter, and use a `key` parameter to specify the basis of your sort. For example, instead of a normal sort, we can reverse sort by the absolute values as given below:

```
my_list = [2, 10, 1, -5, 22]

# Sorted reversely on basis of absolute value
my_list_sorted_abs = sorted(my_list, key=abs, reverse=True)

print(my_list_sorted_abs)
```

The `zip` function

Lastly, a slightly more complicated function: `zip`. You can do a lot of useful things with `zip`, but here are 2 common use cases:

- Combining two lists into a list of tuples
- Breaking a tuple into two lists

Combining two lists into a tuple

The `zip` actually returns a *generator*, so we have to wrap it in `list()` to print it. This would not be necessary if you wanted to loop over it though, because generators are iterable:

```
list_1 = [1, 2, 3]  # create your first list
list_2 = ['x', 'y', 'z']  # create your second list

print(list(zip(list_1, list_2)))  #combine and print
```

Breaking a tuple into two lists

This function can break a tuple into two lists, exactly the reverse of the above use case.

```
pairs = [('x', 1), ('y', 2), ('z', 3)]  # a list of tuples
letters, numbers = zip(*pairs)  # break into two lists
```

```
print(letters)  # print the first values of the tuples
print(numbers)  # print the second values of the tuples
```

Now that we have discussed the fundamentals of the standard Python library, we will explore our first external Python library.

Numpy an External Library

This lesson introduces an external numpy library by discussing in detail how numpy provides support to handle single or multidimensional arrays and different functionalities.

WE'LL COVER THE FOLLOWING

- Handling arrays with NumPy
  - Single dimensional arrays
  - Two dimensional arrays
- Important functionalities
  - Calculating dot product
  - Generating random values
  - Sampling the data
  - Randomly shuffling values

Handling arrays with NumPy

Single dimensional arrays

For the sake of simplicity, you can think of Numpy as a fast and efficient library for storing and manipulating data - usually through array data structures (which are similar to Python lists). You can find all the documentation for Numpy here. Numpy arrays are different from standard Python lists in many ways; notably, they are faster, take up less space, and have more functionality. It is important to note, though, that these arrays are of a fixed size and type that you define at creation. You cannot infinitely append new values as you might with a list.

Let's see some of the ways you can use arrays in action.

```
import numpy as np

# This creates our array
```

```
np_array = np.array([5, 10, 15, 20, 25, 30])
print("--0--")


# Gets the unique values
print(np.unique(np_array))
print("--1--")

# Calculates the standard deviation
print(np.std(np_array))
print("--2--")

# Calculates the maximum
print(np_array.max())
print("--3--")

# Squares each value in the array
print(np_array ** 2)
print("--4--")

# Adds the arrays together element wise
print(np_array + np_array)
print("--5--")

# The sum of the squares of the elements
print(np.sum(np_array ** 2))
print("--6--")

# Gives you the shape: (rows, columns)
print(np_array.shape)
```

Two dimensional arrays

You will also want to understand how to use 2D arrays, arrays that have both *rows* and *columns*. This is typically how you would represent a **matrix** of data using `numpy`.

```
import numpy as np

# Create 2d array
print("--0--")
np_2d_array = np.array([[1,2,3],
                        [4,5,6]])
print(np_2d_array)

# Calculate the transpose, which is when you swap the columns and rows.
print("--1--")
np_2d_array_T = np_2d_array.T
print(np_2d_array_T)

# Print the shape of the array as (number of rows, number of columns)
print("--3--")
print(np_2d_array.shape)
```

```
# Access elements in the 2d array by index.
# First index is the row number
# Second index is the column number

# Index numbers start from 0
print("--4--")
print(np_2d_array[1,1])
print(np_2d_array[0,2])
```

Important functionalities

Calculating dot product

A useful `numpy` function is the ability to calculate the *dot product* of two vectors. If you need a refresher on what a dot product is, you can check out the Wikipedia page. Here is how you do it with code:

```
import numpy as np
np_array = np.array([5, 10, 15, 20, 25, 30])
dot_product = np.dot(np_array, np_array)
print(dot_product)
```

Generating random values

`Numpy` has some excellent functions to help you randomly sample your data.

First, the `rand()` function returns a random number or numbers between 0 and 1. The number of random values returned depends on the shape provided to the function. If no shape is given, just a single number is returned.

```
import numpy as np

# Generage a single random number in range [0,1)
print("--0--")
print(np.random.rand())

# Generate a matrix of random numbers in range [0,1) with shape (3,2)
print("--1--")
print(np.random.rand(3,2))
```

You can also use `randint()` to return random integers from low (inclusive) to high (exclusive). You pass the low and high values as parameters to the function as well as the size of the output.

```
import numpy as np

# Low=5, High=15, Size=2. Generate 2 values between 5 and 15 (exclusive)
print("--0--")
print(np.random.randint(5, 15, 2))

# Low=5, High=15, Size=(3,2). Generate a matrix of shape (3,2) with values between 5 and 15 (
print("--1--")
print(np.random.randint(5, 15, (3,2)))
```

Sampling the data

Once you have some data, it can be useful to sample from it.

As a refresher, sampling is a way to take a smaller group from a population. For example, you might create a random sample from the U.S. population randomly knocking on 10 doors in the U.S. That would be a sample size of 10.

The `choice()` function allows you to pass an array, specify how many values to sample, and decide whether sampling should be done with or without replacement. Sampling without replacement means the same value can't be sampled more than once.

```
import numpy as np
array = np.array([1,2,3,4,5])

# Sample 10 data points with replacement.
print("--0--")
print(np.random.choice(array, 10, replace=True))

# Sample 3 data points without replacement.
print("--1--")
print(np.random.choice(array, 3, replace=False))
```

Randomly shuffling values

Randomly Shuffling Values

Sometimes, it can be helpful to randomly shuffle your array. `Numpy` has a `shuffle()` function that does just that.

> **Note:** The shuffle happens in place, so you don't have to reassign the array to a new value.

```
import numpy as np

x = [1,2,3,4,5]  # Create a list of 5 elements
np.random.shuffle(x)  # Randomly shuffle the order of the elements in the list

print(x)
```

Also, when using randomness, if you want the random selections to be the same every time, you need to set a **seed**. This is possible using the function: `numpy.random.seed()`. We have to supply the function with an integer to *seed* the randomness, so the results can be replicated as long as the same seed is used.

```
import numpy as np
np.random.seed(42)
```

If we put the above code at the beginning of our script, it will ensure the randomness is always the same and tied to the seed 42.

We will end our discussion on Python by examining the library **Scipy**, in the next lesson.

Scipy an External Library

This lesson introduces an external scipy library by discussing in detail how scipy provides support to handle statistics and probabilistic functionalities.

WE'LL COVER THE FOLLOWING

- Calculating correlations
- Generating samples from distributions
  - Normal distribution
  - Probability density function
  - Cumulative distribution function
- Calculating descriptive statistics

Calculating correlations

Scipy is a Python library for scientific computing. Scipy and Numpy are the core libraries that Pandas is built upon. We will discuss Pandas later in the course, but having an understanding of Scipy and Numpy before discussing Pandas is useful.

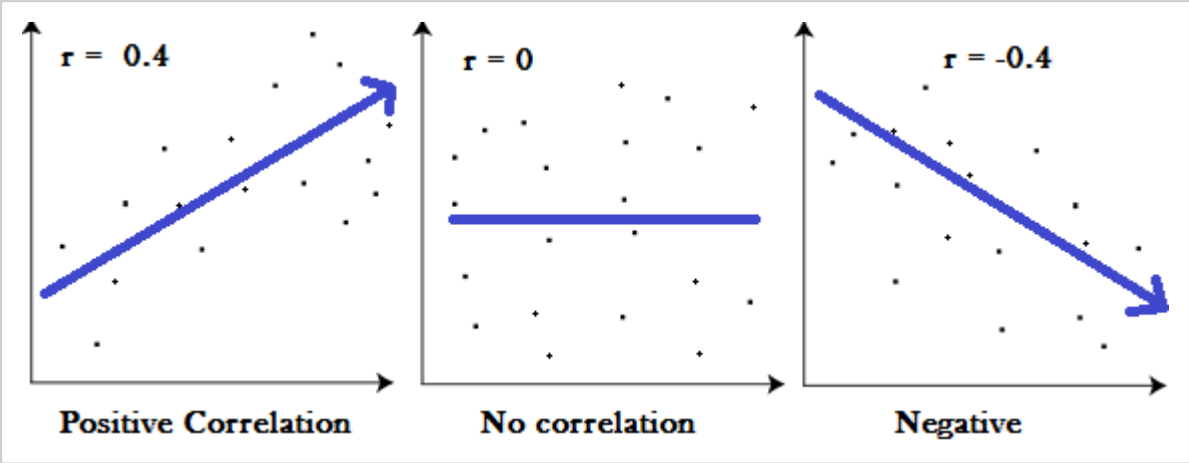
A **correlation** is a numerical measure of the statistical relationship between two variables. For us, those variables will usually be two columns of data, for example, the temperature outside and the likelihood of rain.

One way to calculate the correlation between two vectors of data is with **Pearson's r-value**. This value ranges between -1 and 1. Where -1 means there is a total negative correlation, 0 means no correlation, and 1 means total positive.

> **Note**: these are all linear correlations.

In the image below, you can see a graphical representation of correlation

In the image below, you can see a graphical representation of correlation. Source.



An example of a positive correlation might be height and weight as generally taller people weight more. For a negative correlation, the temperature outside and heating costs since as it is warmer outside you run your heater less. For no correlation, the number of shoes you own and your IQ (clearly two completely unrelated entities).

You can find the mathematical formula on the Wikipedia page.

Here is how you would do it with `scipy`:

```
from scipy import stats
import numpy as np

array_1 = np.array([1,2,3,4,5,6])  # Create a numpy array from a list
array_2 = array_1  # Create another array with the same values

print(stats.pearsonr(array_1, array_2))  # Calculate the correlation which will be 1 since th
```

Since the arrays are the same, we expect there to be a perfectly positive correlation. The result is a tuple containing two values. The first value is the value of correlation. It's **1**, which means it's a positive correlation. Where the second value in the tuple is the *p-value*.

Generating samples from distributions

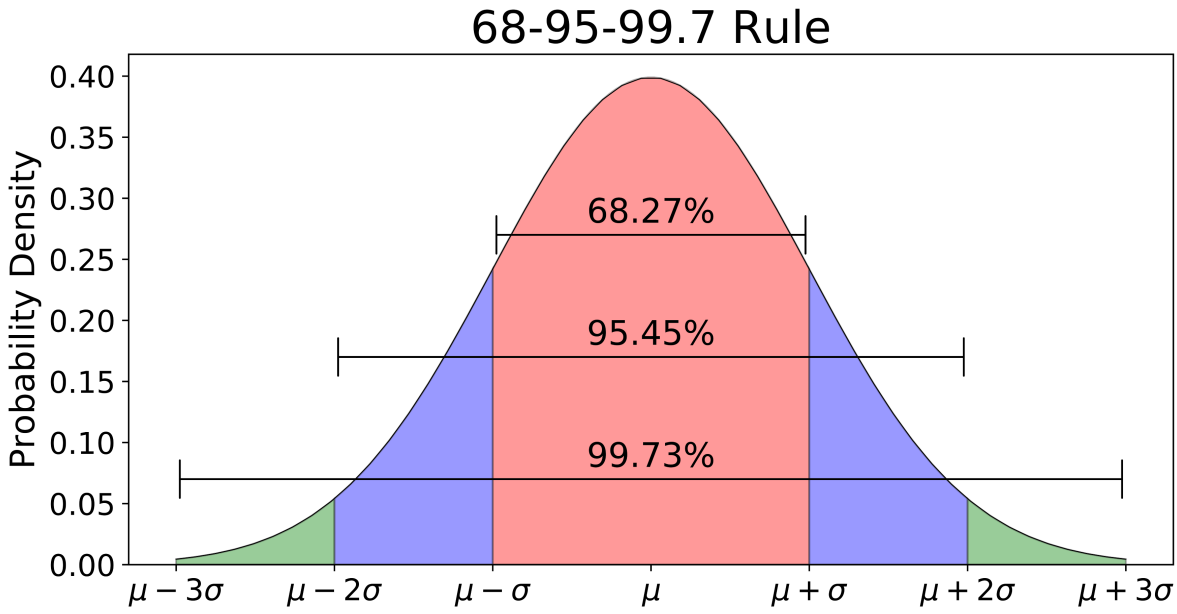
Scipy supports many distributions, we will look at the normal distribution.

Normal distribution

The normal distribution is one that is symmetric around the mean and for which values closer to the mean are more common. It is your standard “bell curve” distribution. See the image below (source).

You can see in the image the probabilities of being within 1, 2, and 3 standard deviations from the mean.

Many natural phenomena have been found to be normally distributed. For example, height.



Say you needed to generate data from a normal distribution with a mean of 0 and a standard deviation of 10; here is how you would do that:

```
from scipy import stats

x = stats.norm.rvs(loc=0, scale=10, size=10)  # Generate 10 values randomly sampled from a no

print(x)
```

The code above uses the `loc` parameter as the *mean*, the `scale` as the *standard deviation*, and the `size` as the *number of samples* to return. If you sampled enough data points and plotted the results, you would see a normal distribution centered around 0 with a standard deviation of 10.

Probability density function

Another common operation on distributions is to calculate the **probability density function**. This function will give you the *relative* likelihood that you would sample a particular value. Let's look at a few:

```
from scipy import stats

p1 = stats.norm.pdf(x=-100, loc=0, scale=10)  # Get probability of sampling a value of -100
p2 = stats.norm.pdf(x=0, loc=0, scale=10)     # Get probability of sampling a value of 0

print(p1)
print(p2)
```

Above we see that relatively it is much more unlikely to sample the value of **-100** (the parameter `x`) at **line 2**, from our distribution than a value of **0** (at **line 3**). This makes sense as our normal distribution is centered on 0 and has a standard deviation of 10.

Cumulative distribution function

Another common calculation is the **cumulative distribution function**, the probability of sampling a value less than or equal to `x`.

```
from scipy import stats

p1 = stats.norm.cdf(x=0, loc=0, scale=10)  # Get probability of sampling a value less than or

print(p1)
```

We can see that the *cumulative distribution function* with `x=0` is **0.5** because **0** is the *mean* and with a normal distribution, half of the data is less than or equal to the mean.

Calculating descriptive statistics

Lastly, if you have an array of values in Scipy, you can use the `describe()` function to calculate multiple descriptive statistics for your array. Consider

the following program:

```
from scipy import stats

print(stats.describe(stats.norm.rvs(loc=0, scale=1, size=500)))  # Calculate descriptive stat
```

As you can see above, you get the `number of observations`, the `min`, the `max`, the `mean`, the `variance`, the `skewness`, and the `kurtosis` of your array.

> **Note**: The `variance` is the square of the standard deviation. The `skewness` is a measure of the asymmetry of the distribution. The `kurtosis` is a measure of the "tailedness" of the distribution. A large value usually means there are more outliers.

Now that you have enough information on `numpy` and `scipy` library, the next lesson brings you a challenge to solve.

Exercise: Using Numpy and Scipy

This lesson provides a few exercises to test your understanding of the Python basics for analytics.

WE'LL COVER THE FOLLOWING

- Numpy
- Scipy

Numpy

Create a function which takes that `numpy` 1-D array as input and returns the following (in the same order as listed):

- **Max** - Maximum value in the array
- **Std** - Measure of variation between the elements of an array
- **Sum** - Value obtained as a result of adding all the elements of an array
- **Dot product** - Inner product of the vectors

Try to implement the function below. Feel free to view the solution, after giving it a few shots. Good Luck!

```
import numpy as np

def perform_calculations(array):
  return 0, 0, 0, 0 # Replace with max, std, sum, and dot product
```

Scipy

Create a function that takes in two `numpy` 1-D arrays and returns the `correlation` and `p-value` as a *tuple*.

Try to implement the function below. Feel free to view the solution, after

Try to implement the function below. Feel free to view the solution, after giving it a few shots. Good Luck!

```
from scipy import stats
import numpy as np

def correlation(array1, array2):
  return (0,0) # Replace with a tuple containing (correlation, p-value)
```

We hope that you were able to solve the challenges. The next lesson brings you the solutions to the above challenges.

Solution Review: Using Numpy and Scipy

This lesson provides the solutions to the previous challenges.

WE'LL COVER THE FOLLOWING

- Numpy
- Scipy

Numpy

```
import numpy as np  # importing numpy module

def perform_calculations(array):

    # returning max, std, sum, and dot product
    return np.max(array), np.std(array), np.sum(array), np.dot(array, array)

# calling the function and printing result
print(perform_calculations(np.random.rand(5)))
```

According to the problem statement, we needed these *four* values using *one* `numpy` 1-D array as an output: **max**, **std**, **sum**, and **dot product**. In the code above, at **line 1** we imported the `numpy` module for this purpose. Next, we implemented the function `perform_calculations()`.

Look at its header at **line 3**. It takes one argument, `array` as an input. At **line 6**, we called *four* built-in functions for required outputs:

- `numpy.max(array)` : Returns the *maximum* value from the `array`
- `numpy.std(array)` : Returns the *standard deviation* among the values of `array`.
- `numpy.sum(array)` : Returns the *sum* of all the values of `array`.
- `numpy.dot(array, array)` : Returns the *dot product* of `array` with itself.

- `numpy.dot(array)` : Returns the *dot product* of `array` with itself.

Now, look at **line 9** where we are calling the function `perform_calculations(array)`. We are generating an array of **five** random values, using `np.random.rand(5)`. It returns an array which is passed to the function `perform_calculations(array)`, and **max**, **std**, **sum**, and **dot product** are printed at the end.

Scipy

```
from scipy import stats  #importing scipy module
import numpy as np       #importing numpy module

def correlation(array1, array2):

  # returning correlation and p-value as a tuple
  return stats.pearsonr(array1, array2)

# calling the function and printing result
print(correlation(np.random.rand(5),np.random.rand(5)))
```

According to the problem statement, we needed these *two* values using *two* `numpy` 1-D array as an output: **correlation**, and **p-value**. In the code above, at **line 1** we imported the `stats` module from `scipy` library for this purpose. Next, we implemented the function `correlation()`.

Look at its header at **line 4**. It takes two argument, `array1` and `array2` as an input. At **line 7**, we called *one* built-in function for the required output:

- `stats.pearsonr(array1, array2)` : Returns the *correlation* and *p-value* value between `array1` and `array2`.

Now, look at **line 10** where we are calling the function `correlation(array1, array2)`. We are generating two arrays each of **five** random values, using `np.random.rand(5)` which are then passed to the function `correlation(array1, array2)`, and **correlation** and **p-value** are printed at the end in the form of a tuple.

That's it about the basics of Python and how to use Python to calculate and gather the main statistics of data. The next chapter explains how to read the

data that is to be analyzed and visualized in the future.

Comma Separated Files

This lesson focuses on CSV type files. It gives a complete explanation about how to read data from CSV files using the Pandas library of Python.

WE'LL COVER THE FOLLOWING

- Introduction to CSV file
- Reading CSV file with Pandas

Introduction to CSV file

Comma-separated files (CSV) are common in machine learning. These files have a **row** of data per line of the file and each line is a *comma-separated list* in which each element is a **column**. Pandas makes it easy to read this data.

Reading CSV file with Pandas

The documentation can be found here. Before reading a CSV file, there are three parameters that should be known:

- `sep` - this defaults to a comma, but we can specify anything we want. For example, CSV format is poor if some of your columns contain commas. A better option might be a |.
- `header` - which row (if any) have the column names.
- `names` - column names to use.

If your CSV is well-formatted where the first row is the column names, then the default parameters should work well.

It is important to note that while it might sound simple to read in a CSV file without Pandas, CSV files are often very messy and reading them appropriately can often consist of handling many edge cases. Pandas module handles many of those edge cases right out of the box and has many parameters that you can change to handle messier CSV files.

parameters that you can change to handle messier CSV files.

Let's see an example with code.

```
import pandas as pd

# Define the column names as a list
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
# Read in the CSV file from the webpage using the defined column names
df = pd.read_csv("adult.data", header=None, names=names)

print(df.head())
```

Let's deconstruct the code. First, we have a CSV of data that lives at this page.

Go to the page, click on **Data Folder** and then select **adult.data** file. It will download the small dataset for you. Once downloaded, open it in Excel or any text editor. You will see rows of data with each column separated by commas.

You will notice that this CSV doesn't have column names. Fortunately, we know what the names should be and supplied them to the `names` parameter at **line 2**. Since Pandas assumes the first row is the `header` (columns), `header=None` was used at **line** 7, so it would read the first row as data.

The data is read into a Pandas dataframe.

A Pandas dataframe is very similar to a matrix of data, but with some additional benefits (usually at the cost of performance). For example, you get named columns and rows as well as the ability to store different types of data in each column. For example, one column could be integers and another text.

You might imagine a dataframe as an Excel sheet of data. For example, if you had a sheet with a column of dates and a column of temperatures on those dates, you could easily represent it as a dataframe.

We can see the first 5 rows of data by calling the `head()` function on the dataframe. In the next section on describing data, we will go more in-depth on how to use a dataframe.

Next, let's look at reading in JSON files.

JSON Files

This lesson focuses on JSON type files. It gives a complete explanation about how to read data from JSON files using the Pandas library of Python.

WE'LL COVER THE FOLLOWING

- Introduction to JSON file
- Reading JSON file with Python

Introduction to JSON file

JSON (**J**ava**S**cript **O**bject **N**otation) is a popular format allowing for a more flexible schema. It is also easy for humans to read and write. A lot of the data sent around the web is transmitted as JSON. Here is an example:

```
{
    "glossary": {
        "title": "example glossary",
        "GlossDiv": {
            "title": "S",
            "GlossList": {
                "GlossEntry": {
                    "ID": "SGML",
                    "SortAs": "SGML",
                    "GlossTerm": "Standard Generalized Markup Language",
                    "Acronym": "SGML",
                    "Abbrev": "ISO 8879:1986",
                    "GlossDef": {
                        "para": "A meta-markup language, used to create ma
rkup languages such as DocBook.",
                        "GlossSeeAlso": ["GML", "XML"]
                    },
                    "GlossSee": "markup"
                }
            }
        }
```

```
    }
}
```

Reading JSON file with Python

Python can easily read these data from strings into dictionaries using the built-in `json` library:

```
import json

## Define the JSON object as a string
json_string = """{
    "glossary": {
        "title": "example glossary",
        "GlossDiv": {
            "title": "S",
            "GlossList": {
                "GlossEntry": {
                    "ID": "SGML",
                    "SortAs": "SGML",
                    "GlossTerm": "Standard Generalized Markup Language",
                    "Acronym": "SGML",
                    "Abbrev": "ISO 8879:1986",
                    "GlossDef": {
                        "para": "A meta-markup language, used to create markup languages such
                        "GlossSeeAlso": ["GML", "XML"]
                    },
                    "GlossSee": "markup"
                }
            }
        }
    }
}"""


# Read the JSON data into Python
json_data = json.loads(json_string)

print(json_data)
```

When your JSON data are in a string, you can use the `loads()` function to read it into a Python dictionary (**line 29**). JSON is the same format as a dictionary in that it consists of key, value pairs of various types. Our JSON object above has a key for `title` and the value is **example glossary**. It also has a key of `GlossList` that has a **dictionary object** for a value that contains its own key-value pairs. You can see how JSON can represent any type of nested schema you would want to have.

Now that you have your JSON object as a dictionary you can access the values

Now that you have your JSON object as a dictionary you can access the values in the same way we discussed in the data structures lesson. If you have a JSON

in a file, you read the data using the `load()` function. Here is how you would do so if you had a file called **data.json**:

```
with open('data.json') as f:
    data = json.load(f)
```

Now that you're familiar with JSON type files, in the next section, we will look at reading in raw text files.

Raw Files

This lesson focuses on raw files. It gives a complete explanation about how to read data from raw files using Python.

WE'LL COVER THE FOLLOWING

- Introduction to raw files
- Reading in raw files with Python

Introduction to raw files

Sometimes you get data in strange formats and you have to roll your own Python code to process the data. Fortunately, doing this is simple.

For this, we will assume that you have data in some type of text file. Each row of data corresponds to a row in your text file.

For example, you might have a file delimited by a pipe (|). It could look something like this:

```
James|22|M
Sarah|31|F
Mindy|25|F
```

Above we have 3 rows of data each with 3 values separated by a pipe (|) - name, age, and gender.

We can create this same file in Python to process using tempfile.

```
import tempfile

tmp = tempfile.NamedTemporaryFile()

# Open the file for writing. And write the data.
with open(tmp.name, 'w') as f:
    f.write("James|22|M\n")
    f.write("Sarah|31|F\n")
```

```
    f.write("Mindy|25|F")

# Read in the data from our file, line by line
with open(tmp.name, "r") as f:
    for line in f:
      print(line)
```

Above, we used `tempfile` (**line 6**) to create a file that contains three rows of data. The **\n** at the end of the first 2 rows of data (**line 7-8**) tells the program to create new lines.

Reading in raw files with Python

The part of the code we care about is:

```
with open(tmp.name, "r") as f:
    for line in f:
      print(line)
```

We use the `open()` command to read the file. We first pass the *name* of the file we want to process and then an **r** which stands for *read*.

Since we used the `with as` functionality in Python, our filehandle is now represented as the variable `f` and is only relevant within this scope. This is good practice for reading files because you don't have to worry about closing the filehandle. Once you have a filehandle, Python makes it very easy to access the lines. The variable `f` is now an iterable where each iterable is a line of the file. Thus, when we do

```
for line in f
```

We are just looping over the lines of our file. The code above only prints out those lines, but since we have direct access to the lines of our file, we can do whatever we want in our processing. For example, here is code that takes only the first value of each row and appends them to a list.

```
import tempfile

tmp = tempfile.NamedTemporaryFile()

# Open the file for writing and write our data
```

```
# Open the file for writing and write our data
with open(tmp.name, 'w') as f:
    f.write("James|22|M\n")
    f.write("Sarah|31|F\n")
    f.write("Mindy|25|F")

first_values = []  # Define a list to store the first values of each row
with open(tmp.name, "r") as f:  # Open the file to read
    for line in f:  # Loop over each line
      row_values = line.split("|")  # Split each line by the | character into a list
      first_values.append(row_values[0])  # Add the first value to our list

print(first_values)
```

That's it on reading files in Python using support from different libraries. In the next lesson, there's a challenge for you to solve.

Exercise: Reading Auto MPG Dataset

This lesson provides an exercise to test your understanding of reading a dataset using Pandas.

WE'LL COVER THE FOLLOWING

- Reading the dataset

Reading the dataset

As an exercise, read Auto MPG Dataset. In this documentation, you will find the **Attribute Information** which lists the column names. Fill whitespace with `_` in *columns' names*.

Click **Data Folder** and then download the file **auto-mpg.data**. The task is fairly simple, you just have to read the dataset, and return its *shape*.

This file is whitespace delimited, so you will need to review the `Pandas` documentation for the `delim_whitespace` parameter.

> **Note**: To read the dataset use **auto-mpg.data** file name.

Try to implement the function below. Feel free to view the solution, after giving it a few shots. Good Luck!

```
import pandas as pd

def read_csv():
    return (0,0) # Update function to read the data into a data frame and return the shape
```

We hope that you were able to solve the challenge. The next lesson brings you the solution to the above challenge.

the solution to the above challenge.

Solution Review: Reading Auto MPG Dataset

This lesson provides the solution to the previous challenge.

WE'LL COVER THE FOLLOWING

- Reading the Dataset

Reading the Dataset

```
import pandas as pd # calling pandas module

def read_csv():

    # Define the column names as a list
    names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "moc
    # Read in the CSV file from the webpage using the defined column names
    df = pd.read_csv("auto-mpg.data", header=None, names=names, delim_whitespace=True)
    return df.shape

# Calling the function and printing the result
print(read_csv())
```

According to the problem statement, we need to get the **shape** of **Auto MPG Dataset** as an output. In the code above, at **line 1**, we imported the `pandas` module for reading the dataset. Next, we implemented the function `read_csv()`.

Look at its header at **line 3**. At **line 6**, we set the name of the *columns* in a list called `names`. We get the name of the columns from the documentation. Notice that `names` hold **9** string values:

- `mpg`
- `cylinders`
- `displacement`

- `horsepower`

- `weight`
- `acceleration`
- `model_year`
- `origin`
- `car_name`

Notice that we replace whitespace with `_` in `model year` and `car name`.

Line 8 is the most important line. We are using a built-in function `read_csv()` from `pandas`. As you are familiar, it takes some arguments as follows:

- `path`: The path to the dataset
- `header`: In case the columns are provided explicitly, it's set to `None`.
- `names`: List of columns' names to be used
- `delim_whitespace`: Specifies whether or not whitespace should be used as a sep

So for `path`, we give the name of the file: **auto-mpg.data**. As we know the columns' names and provided them explicitly to the function as `names=names`, we set `header` as `None`. It was mentioned that there are whitespaces, and we have to use them as sep, so `delim_whitespace` is set to `true`. This built-in function returns a *dataframe* `df` as a result. At **line 9**, we are returning `shape`, a property of the dataframe `df`.

Now, look at **line 12**, where we are calling the function `read_csv()`. It returns a tuple in the form of `(rows, columns)`, which is printed at the end. For this dataset, the function will return `(398, 9)` as there are **398** rows and **9** columns.

That's it about the basics of reading the dataset of different formats using Pandas. The next chapter explains how to describe any dataset in statistical terms.

Statistics and Counts

This lesson discusses principal data processing and data analytics techniques using Pandas.

WE'LL COVER THE FOLLOWING

- Gathering statistics on data
- Finding the data types
  - Converting data types
- Finding unique values
- Grouping the data
- Finding the correlation
- Generating percentiles

In the last chapter, while studying how to read data from CSV files, we used Pandas. Now, we will look deeper at Pandas to process data.

To do so, we will be using the same data set we used when learning about how to read in CSV data. Here are the first 5 rows to refresh your memory.

```
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)
print(train_df.head())
```

Gathering statistics on data

A good place to start is just looking at your data using some pandas functions to better understand what issues there might be. `Describe` will give you counts and some statistics for continuous variables.

```
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)
print(train_df.describe())
```

For all of our numeric values, we now have the `mean`, the `std`, the `min`, the `max`, and a few different `percentiles`.

> **Note**: It is good to remember that the mean value will be influenced more by outliers than the median. Also, you can always square the standard deviation to get the variance.

You may have noticed that some columns are missing. The only columns `describe()` function fetched for us are the ones that hold numeric data.

Finding the data types

To see what types of data we have, we can use the `info()` function:

```
import pandas as pd

# Read in data as explained in reading CSV lesson
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)

print(train_df.info())  # Use the info() function on the dataframe
```

In our dataframe, we have two data types, `object` and `int64`. You can think of an `object` as a string value and `int64` as an integer.

Converting data types

If a column doesn't seem to have the correct type, it is easy to convert it to different types using `.to_()` functions:

- `to_numeric()`
- `to_datetime()`
- `to_string()`

For example:

```
df['numeric_column'] = pd.to_numeric(df['string_column'])
```

You also get the count of non-null values per column and the memory usage of your dataframe from the `info` command used above.

Finding unique values

Another useful step is to look at unique values for columns. Here is an example for the `relationship` column:

```
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)
print(train_df['relationship'].unique())
```

Now we can see all the unique values above and can check the counts of unique values for `relationships`:

```
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)
print(train_df['relationship'].value_counts())
```

This shows us all the unique values for relationships as well as their counts. So, we have a lot of values as **husband** in the `relationship` column, but less **Other-relative**.

Grouping the data

We can also do these types of counts by specific groups by using the `groupby()`

We can also do these types of counts by specific groups by using the `groupby()` function. This function takes a list of columns by which you would like to group your dataframe. It then performs the requested calculations on each group individually and returns the results by group. Here is an example:

```
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)

# Group by relationship and then get the value counts of label with normalization
print(train_df.groupby('relationship')['label'].value_counts(normalize=True))
```

What we did above was group by the variable, `relationship`, and then perform value counts on the variable `label`. For these data, label is whether you make more than **50k**. We can see above that **55%** of *husbands* make more than **50k**. We received *percentages* because we used the `normalize=True` parameter.

You can do many types of calculations on groups using Pandas. For example, here we can see the mean hours worked per week by `workclass`.

```
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)
print(train_df.groupby(['workclass'])['hoursperweek'].mean())
```

From the above, it looks like Federal government workers work more than local workers on average. **Never-worked** average about 28 hours.

Finding the correlation

Another useful statistic is the correlation. If you need a refresher on correlation, please check out Wikipedia. You can calculate all the pair-wise correlations in your dataframe by using the `corr` function.

```
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']

train_df = pd.read_csv("adult.data", header=None, names=names)

# Calculate correlations
print(train_df.corr())
```

We can quickly see that compared to all of the correlations, there is a higher correlation between “hours per week” and “education num”, but it is not very high. You will notice, though since our label is an object, it isn’t included here. Knowing how variables correlate with our label would be useful, so let’s take care of that:

```
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)

# Convert the string label into a value of 1 when >= 50k and 0 otherwise
train_df['label_int'] = train_df.label.apply(lambda x: ">" in x)
print(train_df.corr())
```

There seems to be some decent correlation with the label and education num. One thing to note, though, is that our label is categorical, so correlation doesn’t really apply, our groupby frequencies are probably a better method.

> **Note**: Categorical variables are variables with categories with no intrinsic order. For example, gender.

Also, keep in mind, these are just univariate correlations (between one variable) and don’t account for multi-variate effects (between multiple variables). You can also calculate the correlation using the `scipy` package which has the added benefit of p-values. This was discussed in the “Scipy an External Library” lesson.

Generating percentiles

Lastly, the describe function of Pandas gives some percentiles, but it is easy to add more:

```
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)

# Use the describe function to calculate the percentiles specified
print(train_df.describe(percentiles=[.01,.05,.95,.99]))
```

A percentile is the value below which a given percent of the data falls.

We pass the percentile values we want using the `percentiles` parameter shown above.

In the next lesson, we will take a look at various ways to reshape our dataframe.

Reshaping the Data

This lesson explains the possible ways to reshape and arrange the data using Pandas.

WE'LL COVER THE FOLLOWING

- Pivot Table
- Cross Tab
- Reshape
  - Long to wide format
  - Wide to long format

Pivot Table

Somewhat like Excel, we can pivot our data using pandas pivot_table functionality. To do so, we will use the `pivot_table()` function.

The `values` parameter is the column being used for aggregation, the `index` parameter is for the index values that creates multiple rows, and the `columns` parameter is for the value on which you want to have multiple columns created.

You can also use the `aggfunc` parameter to pass a function with which to aggregate your pivots.

Let's look at an example:

```
import numpy as np
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)

# Pivot the data frame to show by relationship, workclass (rows) and label (columns) the aver
print(pd.pivot_table(train_df, values='hoursperweek', index=['relationship','workclass'],
               columns=['label'], aggfunc=np.mean).round(2))
```

Now we have a table of the average hours per week for a given `relationship`, `workclass`, and `label`.

Cross Tab

`Crosstab` is a nice way to get frequency tables. What you do is pass two columns to the function and you will get the frequency of all the pair-wise combinations of those two variables.

Let's look at an example using **label** and **relationship** as our columns:

```
import numpy as np
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)

# Calculate the frequencies between label and relationship
print(pd.crosstab(train_df['label'], train_df.relationship))
```

We now have counts broken down by **relationship** and **label**. The first parameter is for the *rows*, and the second is for the *columns*. We can also normalize the results using the `normalize=True` parameter.

```
import numpy as np
import pandas as pd
names = ['age', 'workclass', 'fnlwgt', 'education', 'educationnum', 'maritalstatus', 'occupat
        'sex', 'capitalgain', 'capitalloss', 'hoursperweek', 'nativecountry', 'label']
train_df = pd.read_csv("adult.data", header=None, names=names)

# Crosstab with normalized outputs
print(pd.crosstab(train_df['label'], train_df.relationship, normalize=True))
```

Reshape

With Pandas, you can use `pivot()` to reshape your data. To illustrate this concept, I will use code from this post to create a dataframe in a long format.

```
import pandas.util.testing as tm; tm.N = 3
import numpy as np
import pandas as pd

def unpivot(frame):
    N, K = frame.shape
    data = {'value' : frame.values.ravel('F'),
            'variable' : np.asarray(frame.columns).repeat(N),
            'date' : np.tile(np.asarray(frame.index), K)}
    return pd.DataFrame(data, columns=['date', 'variable', 'value'])
df = unpivot(tm.makeTimeDataFrame())
print(df)
```

As you can see from the output above, a long format is where you have a variable which acts like a potential column. In this example, **variable** has values *A*, *B*, and *C*. This is a long format. To make it a wide format, we would make columns *A*, *B*, and *C* and remove the *variable* column.

Long to wide format

Here is how we would covert this long format dataframe to a wide format:

```
import pandas.util.testing as tm; tm.N = 3
import numpy as np
import pandas as pd

def unpivot(frame):
    N, K = frame.shape
    data = {'value' : frame.values.ravel('F'),
            'variable' : np.asarray(frame.columns).repeat(N),
            'date' : np.tile(np.asarray(frame.index), K)}
    return pd.DataFrame(data, columns=['date', 'variable', 'value'])
df = unpivot(tm.makeTimeDataFrame())

# Use pivot to keep date as the index and value as the values, but use the vaiable column to
df_pivot = df.pivot(index='date', columns='variable', values='value')
print(df_pivot)
```

We use the `pivot()` function. The `index` parameter is the column we want for the index of our new dataframe. The `columns` variable is for the column for which we want to use the unique values to create new columns. The `values` parameter is which column we want to use to populate the values of these new columns.

Wide to long format

To convert the format form wide back to long, Pandas provide us with the functionality of unstacking via function `unstack()`. This function (in this situation) will stack our columns back into a single variable.

Let's see an example below:

```
import pandas.util.testing as tm; tm.N = 3
import numpy as np
import pandas as pd

# Create long dataframe
def unpivot(frame):
    N, K = frame.shape
    data = {'value' : frame.values.ravel('F'),
            'variable' : np.asarray(frame.columns).repeat(N),
            'date' : np.tile(np.asarray(frame.index), K)}
    return pd.DataFrame(data, columns=['date', 'variable', 'value'])
df = unpivot(tm.makeTimeDataFrame())

# Convert to wide format
df_pivot = df.pivot(index='date', columns='variable', values='value')

# Convert back to long format
print(df_pivot.unstack())
```

Now that you are familiar with techniques to gather statistics on data and to arrange it the way you want to, the next lesson brings a challenge for you to solve.

Exercise: Group By Aggregations

This exercise will test your abilitiy to do group by aggregations on a pandas data frame

WE'LL COVER THE FOLLOWING

- Group by aggregations

Group by aggregations

We'll use the same function we created in the Reading Data exercise to read in the `pandas` data frame.

We will create an additional function `group_aggregation` which will take a data frame, a group variable, and an aggregation variable. The function will group by the group variable and calculate the mean of the aggregation variable for each group. Your function will return the grouped data frame.

Try to implement the function below. Feel free to view the solution, after giving it a few shots. Good Luck!

```
import pandas as pd
def read_csv():
    # Define the column names as a list
    names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "moc
    # Read in the CSV file from the webpage using the defined column names
    df = pd.read_csv("auto-mpg.data", header=None, names=names, delim_whitespace=True)
    return df

df = read_csv()

def group_aggregation(df, group_var, agg_var):
    return 0 # Update function to return the grouped data frame
```

We hope that you were able to solve the challenge. The next lesson brings you the solution to the above challenge.

the solution to the above challenge.

Solution Review: Group By Aggregations

This lesson provides the solution to the previous challenge.

WE'LL COVER THE FOLLOWING

- Group by aggregations

Group by aggregations

```
import pandas as pd

# Loading dataset
def read_csv():
    # Define the column names as a list
    names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "moc
    # Read in the CSV file from the webpage using the defined column names
    df = pd.read_csv("auto-mpg.data", header=None, names=names, delim_whitespace=True)
    return df

# Describing data
def group_aggregation(df, group_var, agg_var):

    # Grouping the data and taking mean
    grouped_df = df.groupby([group_var])[agg_var].mean()
    return grouped_df

# Calling the function
print(group_aggregation(read_csv(),"cylinders","mpg"))
```

According to the problem statement, we need to group the **Auto MPG Dataset** on the basis of a column. Then we have to calculate the mean of the grouped data according to another column. Before doing it, we have to read the data first. There is no need to explain how to read the data, as we studied that in detail previously. Dataset is read from **line 4** to **line 9**.

Moving towards the main implementation, look at the header of the `group_aggregation(df, group_var, agg_var)` function. It takes *three* arguments

`group_aggregation(df, group_var, agg_var)` function. It takes three arguments as input:

- `df`: A dataframe containing the dataset in the form of a matrix.
- `group_var`: A variable that will group the data
- `agg_var`: A variable to aggregate or describe the grouped data with statistics

Line 15 is the most important line. We are using a built-in function `groupby()` on `df` which takes *one* arguments: `group_var`. It will return the data grouped according to `group_var` column. Next, we are calculating the `mean` of the grouped data with the `agg_var` column. Then at **line 16**, we are returning the result.

At **line 19** we are calling the function `group_aggregation(read_csv(),"cylinders","mpg"))`. First control will transfer to `read_csv()` at **line 4** and we'll get a dataframe.

The next argument is `cylinders`. According to the dataset we have *five* different values for `cylinders`: **3**,**4**,**5**,**6**, and **8**. So the dataframe `df` will be grouped into *five* groups, each group represented by a different cylinder count. In simple words, all cars having **3** cylinders will form one group. Similarly, all cars having **4** cylinders will form another group, and so on.

The last argument is `mpg`. According to the dataset, `mpg` holds continuous values. According to implementation, the *mean* of `mpg` for all the cars (of the same group) will be returned and printed.

According to the result, you will notice:

- The average `mpg` of cars having **3** `cylinders` in total is **20.550000**.
- The average `mpg` of cars having **4** `cylinders` in total is **29.286765**.
- The average `mpg` of cars having **5** `cylinders` in total is **27.366667**.
- The average `mpg` of cars having **6** `cylinders` in total is **19.985714**.
- The average `mpg` of cars having **8** `cylinders` in total is **14.963107**.

That's it about analyzing the dataset with different statistical techniques using Pandas. The next chapter explains how to clean a dataset.

Missing Data

This lesson focuses on missing data by explaining some techniques to clean such data from our dataset.

WE'LL COVER THE FOLLOWING

- What is missing data?
- Filling missing data
  - Using a statistical value
  - Using a model
- Tracking and dropping missing data

What is missing data?

So, you have read in your data only to discover that some values are missing! What do you do?

First, you should try to understand why your data points are missing. Are they missing at random or not? Data that is missing at random could be removed with a large enough dataset. If your data is missing for a reason, perhaps missing means zero? Or is it a strong signal of a sensor malfunction? These non-random missing values should be fixed or leveraged. For example, set them to zero if they should be zero.

Second, you should consider how much data you have relative to how many rows have missing data. If you have 1 million data points with 10 points missing at random, you are probably okay to drop those rows. On the other hand, if you have 300 data points with 100 are missing, then you should probably be hesitant about removing 1/3 of your data. Assuming you can't find an underlying reason for the missing data, how do you handle them?

Filling missing data

There are a few ways to fill in missing values.

Using a statistical value

Fill in a column's missing values with a **statistical value** such as the mean, median, or mode. This has the advantage of being simple and easy to do but also introduces a smaller variance within your column than otherwise would not be present. High variance within a column is usually beneficial (assuming it is real) to a machine learning model. Also, it might not make a lot of sense. For example, imagine a data set of heights and weights with missing weights. You then fill all the missing weights with the average regardless of height. We know that height and weight are correlated, but we are not taking advantage of that knowledge.

Let's look at a simple example of our first idea of filling in missing values with a statistical value.

```
import numpy as np
import pandas as pd
pd_series = pd.Series([5, 10, np.nan, 15, 20, np.nan, 25, 50, np.nan])
print("Average of non-missing values: {0}".format(pd_series.mean()))
pd_series = pd_series.fillna(pd_series.mean())
print(pd_series)
```

Above, we created a Pandas series, which is just a single column of a Pandas dataframe. Within our series, we added 3 missing values with `np.nan`.

Pandas allows you to easily fill in missing values with the `fillna()` function. We pass this function the value we want to use. In this example, we pass the mean of the values that are not missing.

Using a model

Use a model to learn values that seem reasonable for missing values. Basically, leverage the data that is not missing to learn what the missing variable might be. In our height and weight example, you could use height to predict weight when not missing and then apply that model to heights with missing weights to fill in your missing data. A popular algorithm for this is K-Nearest Neighbors. For some more ideas on how to deal with missing data, take a look at this article.

Tracking and dropping missing data

Pandas also makes it easy to **drop** missing values using the `dropna()` function:

```
import numpy as np
import pandas as pd

# Create a series with missing data
pd_series = pd.Series([5, 10, np.nan, 15, 20, np.nan, 25, 50, np.nan])
# Drop rows with missing data
pd_series = pd_series.dropna()
print(pd_series)
```

Or you can find which values are missing with the `isnull()` function.

```
import numpy as np
import pandas as pd

# Create a series with missing values
pd_series = pd.Series([5, 10, np.nan, 15, 20, np.nan, 25, 50, np.nan])
# Show which rows are missing
print(pd_series.isnull())
```

You are returned a new series for which a **True** value means the actual value is missing and a **False** value means there is an actual value.

Now that you're familiar with missing data, in the next lesson, you'll be studying how to handle data that varies significantly from other observations.

Outliers

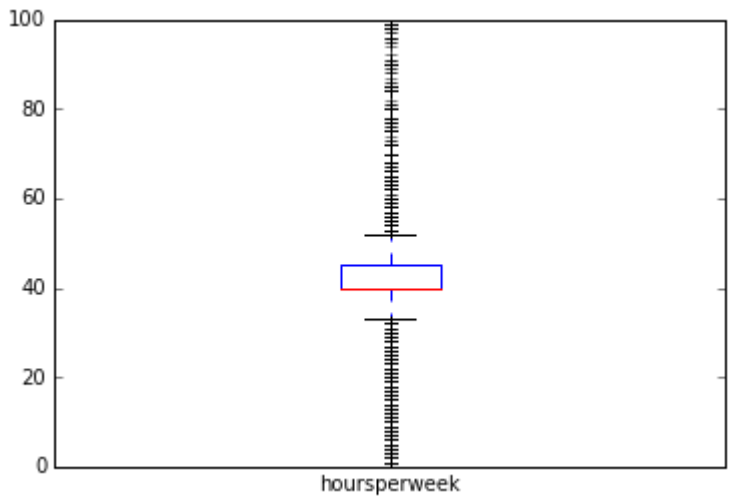
This lesson explains what are outliers, why they happen and how to remove them.

WE'LL COVER THE FOLLOWING

- What is an outlier?
- Detection of an outlier
- Removal of an outlier

What is an outlier?

Another area of cleaning can be dealing with outliers. First off, how do you define an outlier? This can require domain knowledge as well as other information, but a simple way to start is by taking a look at box plots:



Box Plot of Hours Per Week

The above plot was calculated with this command:

```
bbox = train_df['hoursperweek'].plot(kind="box")
```

Detection of an outlier

Here, anything outside the “whiskers” could be considered an outlier. As a refresher, the “whiskers” are the lines sticking out from the box and are 1.5 times the interquartile range. The interquartile range is the distance between the 25th and 75th percentiles.

Here is some code that will calculate the necessary cutoff numbers for all your number variables in a dataframe:

```
q_df = train_df.quantile([.25, .75])
q_df.loc['iqr'] = q_df.loc[0.75] - q_df.loc[0.25]
q_df.loc['whisker_length'] = 1.5 * q_df.loc['iqr']
q_df.loc['max_whisker'] = q_df.loc['whisker_length'] + q_df.loc[0.75]
q_df.loc['min_whisker'] = q_df.loc[0.25] - q_df.loc['whisker_length']
q_df
```

This code assumes your data are in a dataframe called train_df. The first line calculates the values of the 25th and 75th percentiles. It then takes the difference between the two to get the interquartile range (iqr). We get the length of the whiskers by multiplying the iqr by 1.5 and then calculate the min and max whisker value by subtracting and adding the whisker length from the 25th and 75th percentile values. Your output from this code will look something like this:

	age	fnlwgt	educationnum	capitalgain	capitalloss	hoursperweek
0.25	28.0	117827.0	9.0	0.0	0.0	40.0
0.75	48.0	237051.0	12.0	0.0	0.0	45.0
iqr	20.0	119224.0	3.0	0.0	0.0	5.0
whisker_length	30.0	178836.0	4.5	0.0	0.0	7.5
max_whisker	78.0	415887.0	16.5	0.0	0.0	52.5
min_whisker	-2.0	-61009.0	4.5	0.0	0.0	32.5

In this example, we can look more closely at the numbers for “age.” If we use the idea that any value outside of the min or max whisker values is an outlier, then any age greater than 78 or less than -2 would be considered an outlier.

Another simple way to try and identify outliers is by taking some multiple of the standard deviation of your values, usually 2 or 3 times. For example, if the standard deviation of age were 13 and the mean were 30, and you decided anything 3 times the standard deviation could be an outlier, then any age

greater than 69 (13x3 + 30) or less than -9 (30 - 13x3) would be outliers.

For age, we can also use domain knowledge and know that anything less than 0 and greater than 130 are clearly outliers.

All this being said, there are many ways to detect outliers and these are some of the simplest first attempts you could take. These methods look at the feature by itself, sometimes your outliers are multi-variate, a weight by itself might not be an outlier, but for a given height, it is. Here is a post that deals with some more advanced ways of detecting these:

https://www.kdnuggets.com/2017/01/3-methods-deal-outliers.html

Removal of an outlier

Once you think you have found some outliers, though, what should you do?

Like missing data, you need to ask yourself, why might these outliers be here? If they are actually part of your data, then likely you will want to keep them. By removing them, you would skew your training away from the truth. If they are erroneous, for example, ages over 500, you can feel safe dropping those rows if you have enough data or perhaps input a better value. Or even better, try to discover how these bad values came to be and fix them. It could be that your data pipeline broke, but the raw data is still good.

Lastly, it is always good to consider the data generation process. Do your data come from a sample? If so, maybe it isn't very representative, so things that look like outliers in your sample are actually not in the population.

There are many factors to consider and these are only some. The main point is to be thoughtful of your outliers, ignoring or dropping them without thought is usually not the best idea.

Scaling

This lesson explains how scaling helps cleaning the data.

WE'LL COVER THE FOLLOWING

- Introduction to scaling
- Types of scaling
  - Standard scaling
  - Min/Max scaling

Introduction to scaling

The scale of your features matter for many machine learning algorithms. Having income values that range from 100 to 100,000 and ages that range from 0 to 100 can cause issues because of the large difference in scale of these two data columns. To deal with this, it is standard to rescale the data. There are many ways to do this, but the two most common ones are:

- Standard scaling
- Min/Max scaling.

Types of scaling

Standard scaling

Standard scaling subtracts the mean and divides by the standard deviation. This centers the feature on zero with unit variance.

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler
import numpy as np

# Create a matrix of data
data = [[-1, 2],
        [-0.5, 6],
        [0, 10],
        [1, 18]]
```

```
print("Before Standard scaling")
print(np.mean(data, 0))
print(np.std(data, 0))

# Initalize a StandardScaler
standard = StandardScaler()
# Fit and transform the data with the StandardScaler
standard_data = standard.fit_transform(data)

print("After Standard scaling")
print(np.mean(standard_data, 0))
print(np.std(standard_data, 0))
```

In the example above, we created a NumPy array of shape (4,2). We then use the `StandardScaler()` from `sklearn` which will automatically subtract the mean and divide by the standard deviation of each of our columns. This is done with the `fit_transform()` call.

We check that it worked by printing the mean and standard deviation. We can see that both columns now have a mean of 0 and a standard deviation of 1.

Min/Max scaling

Let's look at the same example but instead use the `MinMaxScaler()` from `sklearn`.

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# Create matrix of data
data = [[-1, 2],
        [-0.5, 6],
        [0, 10],
        [1, 18]]

# Initalize MinMaxScaler
min_max = MinMaxScaler()
# Fit and transform the data
min_max_data = min_max.fit_transform(data)

print(np.min(min_max_data, 0))
print(np.max(min_max_data, 0))
print(np.mean(min_max_data, 0))
print(np.std(min_max_data, 0))
```

By default, the `MinMaxScaler()` scales each column to have a min value of **0** and a max value of **1**.

We confirm that our code did this by printing out the min and max values. We also printed out the mean and std of the columns to see that they are no longer 0 and 1 respectively. You can change the min, max range to what you want by passing the following parameters:

```
feature_range=(min, max)
```

That's it about scaling the data. In the next lesson, you'll learn how to handle the data if data is categorical.

Categorical Data

This lesson discusses what is categorical data and how Pandas provide support dealing with it.

WE'LL COVER THE FOLLOWING

- Introduction to categorical data
- Dealing with categorical data
  - Label encoding
  - One-hot encoding

Introduction to categorical data

Sometimes you get categorical data which are variables with a limited and usually fixed number of values. For example, **male** and **female**. Machine learning algorithms need numbers to work, so how do you deal with these? We will discuss two ways:

- Label encoding
- One-hot encoding a.k.a. dummy variables.

Dealing with categorical data

Label encoding

Label encoding works by converting the unique values to a numeric representation. For example, if we have two categories **male** and **female**, we can categorize them as numbers:

- male as **0**
- female **1**

Pandas provides an easy way to do this by using the `category` type.

```
import pandas as pd

# Create series with male and female values

non_categorical_series = pd.Series(['male', 'female', 'male', 'female'])
# Convert the text series to a categorical series
categorical_series = non_categorical_series.astype('category')
# Print the numeric codes for each value
print(categorical_series.cat.codes)
# Print the category names
print(categorical_series.cat.categories)
```

In the code above, we create a Pandas series with text values of either *male* or *female*.

We then cast the column using the `astype()` function and pass the type of `category`. This finds all the unique values in our column and assigns each a unique integer value while still maintaining the string values.

You can get the integer values by adding `.cat.codes` to the end of your category series.

You get the string values by adding `.cat.categories` to the end of your category series.

One-hot encoding

One-hot encoding is similar but creates a new column for each category and fills it with a 1 for each row with that value and zero otherwise.

```
import pandas as pd

# Create series with male and female values
non_categorical_series = pd.Series(['male', 'female', 'male', 'female'])
# Create dummy or one-hot encoded variables
print(pd.get_dummies(non_categorical_series))
```

We see that we just had to use the `get_dummies()` call on our series and it automatically makes new columns for each unique value in our series and fills them with a 1 or 0 as appropriate.

Now that you have a grasp on some ways of cleaning our data, the next lesson

Now that you have a grasp on some ways of cleaning our data, the next lesson brings you a challenge to solve.

Exercise: Cleaning Auto MPG Dataset

This exercise will test a simple way to start investigating potential outliers.

WE'LL COVER THE FOLLOWING

- Detect outliers

Detect outliers

In this exercise, we will use the same data frame as read in from the Reading Data exercise.

We will then create an additional function `outlier_detection` that takes a data frame `df` and returns 2 numbers in a list:

- The 90th percentile for every column
- The 10th percentile for every column

Try to implement the function below. Feel free to view the solution, after giving it a few shots. Good Luck!

```
import pandas as pd
def read_csv():
    # Define the column names as a list
    names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "moc
    # Read in the CSV file from the webpage using the defined column names
    df = pd.read_csv("auto-mpg.data", header=None, names=names, delim_whitespace=True)
    return df

df = read_csv()

def outlier_detection(df):
    return [0,0] # Return 90th and 10th percentiles for numeric_column_name
```

We hope that you were able to solve the challenge. The next lesson brings you

the solution to the above challenge.

Solution Review: Cleaning Auto MPG Dataset

This lesson provides the solution to the previous challenge.

WE'LL COVER THE FOLLOWING

- Cleaning the dataset

Cleaning the dataset

```
import pandas as pd

def read_csv():
    # Define the column names as a list
    names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "moc
    # Read in the CSV file from the webpage using the defined column names
    df = pd.read_csv("auto-mpg.data", header=None, names=names, delim_whitespace=True)
    return df

# Remving outliers from the data
def outlier_detection(df):
    df = df.quantile([.90, .10])
    return df

print(outlier_detection(read_csv()))
```

According to the problem statement, we need to find percentile from the data **Auto MPG Dataset** of all columns. Before doing it, we have to read the data first. There is no need to explain how to read the data, as we studied that in detail previously. Dataset is read from **line 4** to **line 8**.

Moving towards the main implementation, look at the header of the `outlier_detection(df)` function at **line 11**. It takes *one* arguments as input:

- `df`: A dataframe containing the dataset in the form of a matrix.

Line 12 is the most important line. We are using a built-in function

`quantile()` on `df` which takes *one* argument: `[.90, .10]`. We are specifying

the range for quartile that it should be between **0.90** and **0.10**. It will return *two* numbers in a list for each column.

At **line 15** we are calling the function `outlier_detection(read_csv())`. First control will transfer to `read_csv()` at **line 3** and we'll get a dataframe. Then control will go to **line 11** and 90th and 10th percentile will be returned for each column.

For the result, you will notice that percentiles are printed. For example, the 90th and 10th percentile for `mpg` is **34.33** and **14.00** respectively.

That's it about the main concepts regarding cleaning the dataset using Pandas. The next chapter explains how to visualize a dataset using the `seaborn` library.

Introduction to Visualization

This lesson will cover the importance of visualizing data, and why visualization is required in data science as well as common tools.

WE'LL COVER THE FOLLOWING

- The importance of data visualization
- Data visualization tools in Python
- Our dataset

The importance of data visualization

So far, we have looked at understanding data via descriptive statistics and tables. Another useful tool is visualization.

Visualizations of data can provide the following benefits:

- A better understanding of the data
- A more compelling story when explaining the data
- An easier to comprehend medium

Data visualization is a core skill necessary for any analyst or data scientist. Being able to use great visualizations to help tell a data story often significantly adds to the comprehension of others. That added understanding can be the difference in driving a project forward.

Data visualization tools in Python

In Python, two of the most popular tools for visualizing data are **Matplotlib** and **Seaborn**. These are the tools we will focus on, but there are many others including **Bokeh**, **ggpy**, and **D3**.

Matplotlib is sort of the base plotting library in Python. Think of it as a low-level library that allows you to do all sorts of things, but this flexibility can

sometimes make it hard to work with. Matplotlib has been around for a while and sometimes can look a bit dated in style.

Seaborn was created to help deal with some of these issues. It is built on top of Matplotlib and in its own words, “provides a high-level interface for drawing attractive statistical graphics.” For the most part, when possible, I lean towards using Seaborn. When I need more low-level control, I pull in Matplotlib. Since Seaborn is built on top of Matplotlib, it is pretty easy to mix the two.

Our dataset

Before we get started with visualization, I will show how to load in the dataset we will be using:

```
from sklearn.datasets import load_boston
import pandas as pd

# Load the boston dataset from sklearn.datasets
boston_data = load_boston()
# Enter the boston data into a dataframe
boston_df = pd.DataFrame(boston_data.data, columns=boston_data.feature_names)

# Print the first 5 rows to confirm ran correctly
print(boston_df.head())
```

You will notice that we load in the Boston dataset from sklean to use as data to visualize and convert it to a Pandas dataframe in the code above.

Now! Let’s start visualizing some data! We will cover the following types of plots:

- Scatter
- Bar
- Distribution
- Line
- Heat map
- Data aware

- Data aware

Visualization with Scatter Plots

This lesson explains what a scatter plot is, why it is used, and how to visualize data with scatter plot using Python libraries.

WE'LL COVER THE FOLLOWING

- Introduction to scatter plots
- Scatter plots in Python

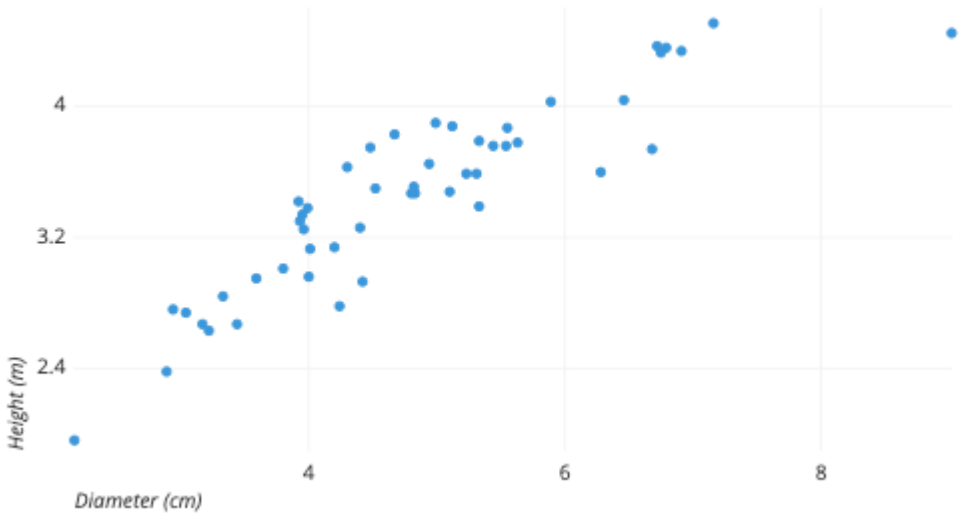
Introduction to scatter plots

First, let's take a look at **scatter plots.**

Scatter plots are great for plotting two variables to visualize how they might correlate and what relationship there might be between the variables.

As an example, look at the scatter plot below.

In this plot, we see a positive relationship between **Height** and **Diameter**. It is easy to see that relationship in a scatter plot by looking for points that are close to the 45-degree line pointing up and to the right.



Scatter plots in Python

Scatter plots in Python

Seaborn makes it easy to create scatter plots using the `lmplot()` function. The key inputs are:

- `x` which is the column name for your x-axis variable
- `y` which is the column name for your y-axis variable
- `data` which is your Pandas dataframe

Let's look at an example:

```
from sklearn.datasets import load_boston
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Set the palette and style to be more minimal
sns.set(style='ticks', palette='Set2')

# Load data as explained in introductory lesson
boston_data = load_boston()
boston_df = pd.DataFrame(boston_data.data, columns=boston_data.feature_names)

# Create the scatter plot
sns.lmplot(x="CRIM", y="NOX", data=boston_df)
# Remove excess chart lines and ticks for a nicer looking plot
sns.despine()
```

You can see in our code that we pass the `x` and `y` variables we want to use from our `data`, which is expected to be a Pandas dataframe that has the x and y values we wish to plot.

By default, it plots a regression line so you can see potential linear relationships more easily. Our variables appear to be somewhat positively correlated. The lighter shade around the line is the 95% confidence interval for our regression line and is calculated using bootstraps of our data. You can see all of the options for this plot here.

Also, we used one of seaborn's styles and palettes. This allows us to control at a high-level how our plots will look. You can learn more about controlling aesthetics here. We chose a style and palette which creates a more minimalistic look. We also called `despine()` after creating the plot to remove excess chart lines and ticks which I believe creates a cleaner look.

That’s how your data can be visualized through a scatter plot. Next, we’ll look at bar plots.

Visualization with Bar Plots

This lesson explains what a bar plot is, why it is used, and how to visualize data with bar plot using Python libraries.

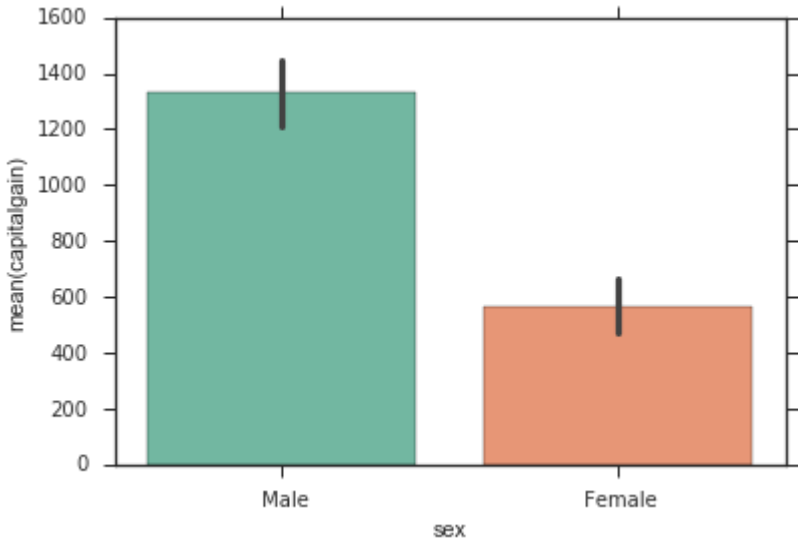
WE'LL COVER THE FOLLOWING

- Introduction to bar plots
- Bar plots in Python

Introduction to bar plots

Bar plots can be useful when comparing categories. For example, the bar plot below compares the capital gain values across gender.

The bar plot allows us to easily see the difference in mean capital gain tax across genders for this dataset.



Bar plots in Python

We can create this bar plot using the `barplot()` command in `Seaborn`.

The first argument to the command is the column to be used for the groups. In the plot above, **gender**.

The second argument is the column to be used for comparison. In the plot above, **capital gain**.

Let's take a look at an example with our Boston data:

```
from sklearn.datasets import load_boston
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Set the palette and style to be more minimal
sns.set(style='ticks', palette='Set2')

# Load data as explained in introductory lesson
boston_data = load_boston()
boston_df = pd.DataFrame(boston_data.data, columns=boston_data.feature_names)

# Only keep for ages 96 and 98.2
boston_df = boston_df[boston_df["AGE"].isin([96, 98.2])]

# Create the bar plot
sns.barplot(boston_df['AGE'], boston_df['NOX'])
# Remove excess chart lines and ticks for a nicer looking plot
sns.despine()
```

In the code, we limit our data to only include homes with `age` of 96 or 98.2 to simplify our example.

The first argument to the `barplot()` command is the `AGE` column and the second argument is the `NOX` column. Thus, the plot shows the average `NOX` value by `AGE`.

Seaborn also plots the error bars (the black lines). The error bars are calculated via bootstrapping which randomly resamples our data with replacement. It then draws 95% error bars which are the 95th and 5th percentiles.

You can see all the options for this plot here.

That's how your data can be visualized through a bar plot. Next, we'll look at a few different ways to visualize distributions.

Visualization with Distributions

This lesson introduces us with types of distributions used to visualize data , and how Python provides support with its libraries.

WE'LL COVER THE FOLLOWING

- Introduction to distributions
- Types of distributions
  - Histogram
  - Box plot
  - Violin plot
  - Joint plot

Introduction to distributions

A probability distribution is a mathematical function that provides the probabilities of the occurrence of different possible outcomes.

For example, you might have a program that returns **1** with a 50% probability and **0** with a 50% probability. Thus, 50% of your probability distribution would be assigned to 1 and 50% to 0.

If you were to plot this expected distribution, you would have two bars of equal height for 1 and 0.

Often, with data you don't know the mathematical function which generated your data, so instead you observe the empirical distribution. You might sample 10 colored balls from a bag and get 2 red, 3 yellow, and 5 green. That would then be your empirical distribution and you could graphical represent it with 3 bars. One of height 2 for red, one of height 3 for yellow, and one of height 5 for green.

Seaborn has a few ways to plot distributions:

- Histograms
- Box plots
- Violin plots
- Joint plots

Types of distributions

Histogram

As explained by Wikipedia, a **histogram** is an estimate of the probability distribution of a continuous variable. To construct a histogram, the first step is to **bin** the range of values—that is, divide the entire range of values into a series of intervals—and then count how many values fall into each interval. The bins are usually specified as consecutive, non-overlapping intervals of a variable.

You create a histogram with the `distplot()` function is Seaborn. You only need to pass one argument which is the continuous variable for which you would like to construct a histogram.

Histograms, though, have a very important parameter - the number of bins. You specify the number of bins with the `bin` parameter. If unspecified, Seaborn tries to find a useful number of bins to use. It is important to remember, though that the more bins, the higher variance your plot will have; the fewer bins, the more bias. Be thoughtful when choosing the number of bins as it might change the way you view your data.

Let's take a look:

```
from sklearn.datasets import load_boston
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Set the palette and style to be more minimal
sns.set(style='ticks', palette='Set2')

# Load data as explained in introductory lesson
boston_data = load_boston()
boston_df = pd.DataFrame(boston_data.data, columns=boston_data.feature_names)

# Create the histogram plot
sns.distplot(boston_df.NOX, kde=False)
# Remove excess chart lines and ticks for a nicer looking plot
sns.despine()
```

Here is our same histogram but with 100 bins:

```
from sklearn.datasets import load_boston
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Set the palette and style to be more minimal
sns.set(style='ticks', palette='Set2')

# Load data as explained in introductory lesson
boston_data = load_boston()
boston_df = pd.DataFrame(boston_data.data, columns=boston_data.feature_names)

# Create the histogram plot
sns.distplot(boston_df.NOX, bins=100, kde=False)
# Remove excess chart lines and ticks for a nicer looking plot
sns.despine()
```

You can see the increased variance by increasing the number of bins.

You will notice we have been setting `kde` =**False**. With this equal to True, it plots the shape of the distribution. We won't go into too much detail about this, but you can learn more here.

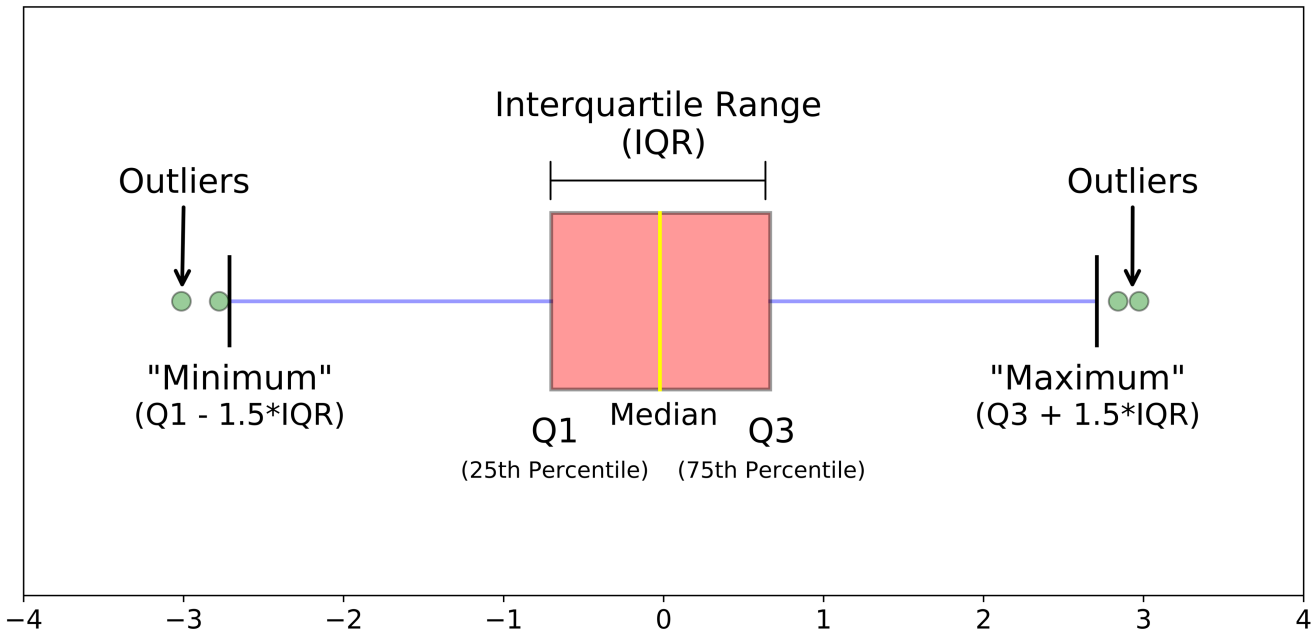
Here is an example:

```
from sklearn.datasets import load_boston
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Set the palette and style to be more minimal
sns.set(style='ticks', palette='Set2')

# Load data as explained in introductory lesson
boston_data = load_boston()
boston_df = pd.DataFrame(boston_data.data, columns=boston_data.feature_names)

# Create the histogram plot
sns.distplot(boston_df.NOX, kde=True)
# Remove excess chart lines and ticks for a nicer looking plot
sns.despine()
```

You can see all the parameters for the `distplot` here.

Box plot

Seaborn also has a box plot. A box plot is a graphical view of summary statistics from a distribution. Here is an example:



The middle line is the median, the right side of the box is the 75th percentile and the left side the 25th percentile. The whiskers are the lines that stick out from the box. Usually these extend out 1.5 times the interquartile range (IQR) where the IQR is the distance between the 75th and 25th percentiles. Any values outside the whiskers are usually shown as dots, and as we know, could be considered outliers.

You can create a box plot using the `boxplot` function from Seaborn. The only argument you usually have to pass is the first one which is the continuous variable you which to represent with a box plot.

Here is an example plotting **NOX**:

```
from sklearn.datasets import load_boston
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Set the palette and style to be more minimal
sns.set(style='ticks', palette='Set2')

# Load data as explained in introductory lesson
```

```
# Load data as explained in introductory lesson
boston_data = load_boston()
boston_df = pd.DataFrame(boston_data.data, columns=boston_data.feature_names)


# Create the box plot
sns.boxplot(boston_df.NOX)
# Remove excess chart lines and ticks for a nicer looking plot
sns.despine()
```

Violin plot

Seaborn has a plot very similar to a box plot called a violin plot. With this plot you have removed the box part of the box plot and replaced it with the kde curve we saw from the `distplot()` function. This can be valuable as it allows you to see the modality of your distribution. For example, below we see that INDUS is bi-modal. In a box plot, we would not have seen that distinction.

> **Note**: kde curves can look smooth with small amounts of data.

You create a violin plot using the `violinplot()` function in Seaborn. You just pass it a dataframe and it will create a violin plot for each column in your dataframe.

I prefer my plots vertically oriented, so I often use the `orient` parameter with a value of **"v"** to do so.

Here is an example with the **INDUS** variable:

```
from sklearn.datasets import load_boston
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Set the palette and style to be more minimal
sns.set(style='ticks', palette='Set2')

# Load data as explained in introductory lesson
boston_data = load_boston()
boston_df = pd.DataFrame(boston_data.data, columns=boston_data.feature_names)

# Create the violin plot
sns.violinplot(boston_df['INDUS'], orient="v")
# Remove excess chart lines and ticks for a nicer looking plot
sns.despine()
```

Joint plot

Lastly, joint plots can plot the histogram of two variables on one plot and also show the scatter plot in the middle. In the top right, you also get the Pearson correlation coefficient between the variables as well as the p-value.

This is done with the `jointplot()` function from Seaborn.

The first parameter is the variable you would like on the x-axis. The second parameter is the variable you would like on the y-axis.

You can also use the `kind` parameter to change how the scatter points are visualized. I like passing a value of **"hex"**.

Here is an example using the **CRIM** and **NOX** variables:

```
from sklearn.datasets import load_boston
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Set the palette and style to be more minimal
sns.set(style='ticks', palette='Set2')

# Load data as explained in introductory lesson
boston_data = load_boston()
boston_df = pd.DataFrame(boston_data.data, columns=boston_data.feature_names)

# Create the violin plot
sns.jointplot(boston_df.CRIM, boston_df.NOX, kind="hex")
# Remove excess chart lines and ticks for a nicer looking plot
sns.despine()
```

That's how your data can be visualized through distributions. Next, we'll look at a few ways to visualize data with the line graphs.

Visualization with Line Graphs

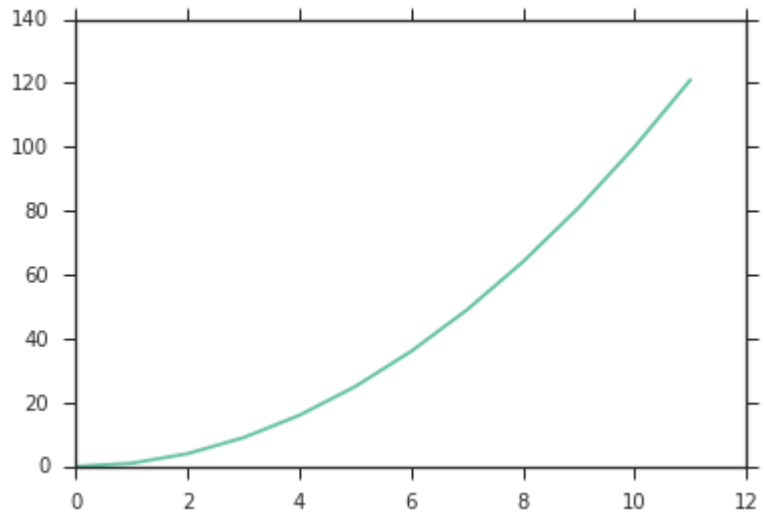
This lesson explains what a line graph is, why it is used, and how to visualize data with a line graph using Python libraries.

WE'LL COVER THE FOLLOWING

- Introduction to line graphs
- Line graphs in Python

Introduction to line graphs

Line graphs are very useful for showing a value over time, such as a stock's price. Typically, one would use a line graph over a scatter plot if there is a connecting component between the values, such as time. Here is an example of a value growing exponentially over time:



Line graphs in Python

The `lineplot()` function of **Seaborn** is what plots the line graph. The *first parameter* is the list or array of x-values and the *second parameter* is the array of y-values.

That's it! Line graphs are simple to implement and can be very effective.

Let's take a look at an example using the flight dataset from Seaborn. This dataset has three columns. **Year**, **Month**, and **Passengers**. The `passengers` column represents the number of flight passengers for that year and month.

```
import seaborn as sns            # importing seaborn functionality
import pandas as pd
import matplotlib.pyplot as plt

flights_long=sns.load_dataset("flights")   # importing dataset

# filtering the dataset to obtain the January records for all years
flights_long=flights_long[flights_long.month == 'January']

#plotting a line graph
plot=sns.lineplot(flights_long.year, flights_long.passengers)
```

The above code loads the dataset, limits the data to only the month of January and then plots the number of passengers for every year as a line plot.

That's how the variations in data along time can be mapped with a line graph. Next, we'll look at heat maps.

Visualization with Heat Maps

This lesson explains what a heat map is, why it is used, and how to visualize data with a heat map using Python libraries.

WE'LL COVER THE FOLLOWING

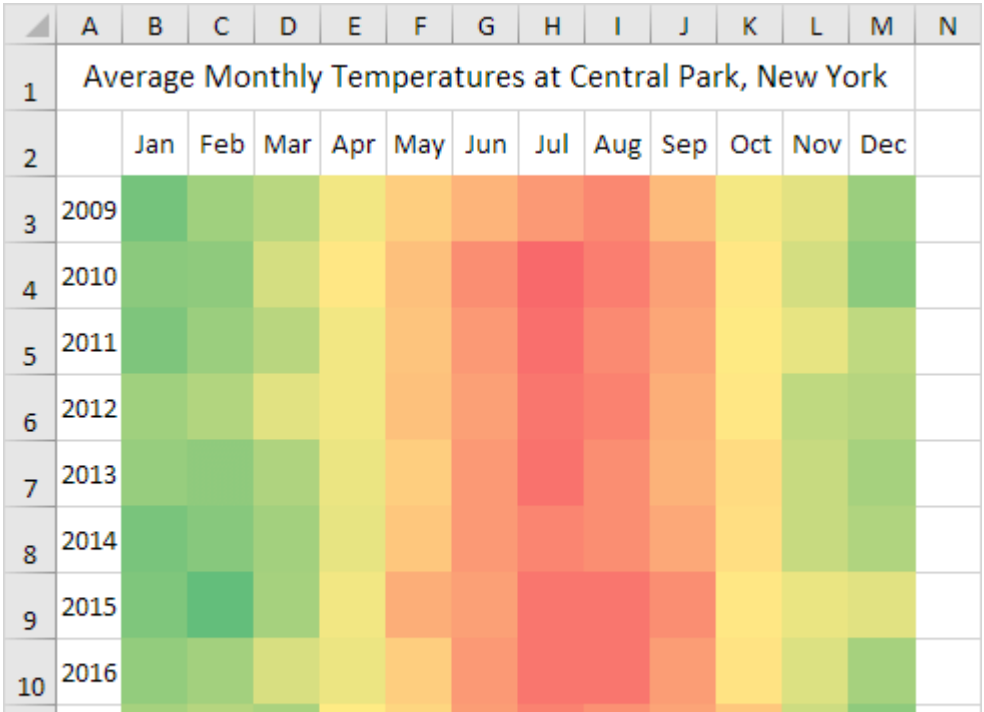
- Introduction to heat maps
- Heat maps in Python

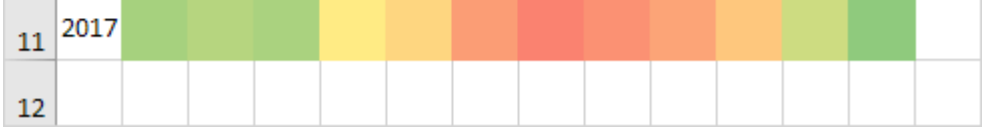
Introduction to heat maps

Heat maps are a plotted data matrix where each value in the matrix is a category.

As an example, consider the heat map shown below.

This heat map represents the average monthly temperatures at Central Park, New York for a given year (**rows**) and month (**columns**). The coloring of the heat map makes it easy to see that the warmest months are consistently from Jun to Sep.





Heat maps in Python

You can create heat maps in Python by using the `heatmap()` function from Seaborn. Some of the core parameters:

- `data` is the first parameter and is your matrix of data represented as a Panda's dataframe.
- `annot` if `True` will plot the actual data values in each cell.
- `fmt` lets you control the string formatting. A value of **"d"** uses a decimal integer.
- `linewidths` lets you set the width of the lines which separate each cell.
- `ax` allows you to pass a custom Matplotlib Axes.
- `cmap` allows you to pick the colormap to use when plotting.

Let's look at an example using the same flight data from the Line Graphs lesson:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
flights_long = sns.load_dataset("flights")
# Pivot the dataset from long to wide format
flights = flights_long.pivot("month", "year", "passengers")
# Create a larger figure size to plot on
f, ax = plt.subplots(figsize=(12, 6))
# Create the heat map
sns.heatmap(flights, annot=True, fmt="d", linewidths=.5, ax=ax, cmap='Blues')
```

Above, we plotted the number of airline passengers by month and year. In our code, we used Matplotlib to adjust the figure size to make it larger, (12,6). That returns a value we called **"ax"** and we pass this to Seaborn with the `ax` parameter. We also use the `annot` = `True` parameter to add the actual values to our cells.

The other option that I want to call out is the `cmap` parameter. This allowed us to adjust the color used for the cells by passing **"Blues"**.

I like heat maps because the color on top of the values allows us to see the trends much easier. We can see over the years the number of passengers has increased, and that July and August are consistently the most popular months. There is a lot of value from a single plot.

That's how the low and high values can be tracked down with heat maps. Next, we'll look at multi-plot grids in Python.

Visualization with Multi-Plot Grids

In this lesson, we will discuss how to use Seaborn to create multi-plot grids.

WE'LL COVER THE FOLLOWING

- Introduction to multi-plot grids
- Multi-plot grids in Python
  - FacetGrid and map
  - PairGrid

Introduction to multi-plot grids

Seaborn also has some multi-plot grid capabilities.

I don't think I could explain it any better than the Seaborn documentation:

When exploring medium-dimensional data, a useful approach is to draw multiple instances of the same plot on different subsets of your dataset. This technique is sometimes called either "lattice" or "trellis" plotting, and it is related to the idea of "small multiples". It allows a viewer to quickly extract a large amount of information about complex data. Matplotlib offers good support for making figures with multiple axes; seaborn builds on top of this to directly link the structure of the plot to the structure of your dataset.

Multi-plot grids in Python

FacetGrid and map

I think the easiest way to understand, is to use an example in Python from the flight's dataset.

First, we will look at the `FacetGrid()` and `map()` combination in Seaborn.

The parameters we will use for `FacetGrid` are:

- `data` which is the first parameter and specifies the dataframe to use.
- `row` which is the column name from your dataframe you want to use as the rows of your grid.
- `col` which is the column name from your dataframe you want to use as the columns of your grid.
- `margin_titles` which if `True`, the titles for the row variable are drawn to the right of the last column.

The parameters we will use for `map` are:

- `func` which is the first argument and is the plotting function you wish to use.
- `args` is the second parameter and is the column name for the variable you wish to plot.
- `color` which allows you to specify the plot color.

Let's take a look:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
flights = sns.load_dataset("flights")
# Subset the data to years >= 1956 to more easily fit on the plot
flights = flights[flights.year >= 1956]

g = sns.FacetGrid(flights, row="year", margin_titles=True)
g.map(plt.plot, "passengers", color="steelblue")
```

First, we created a FacetGrid by passing our dataframe, which in this example is called flights. We then pass the `row` variable (**year**) we want to use in the resulting plot. You can see in our plot above that we have one plot per year. Margin titles tell the plots to add the titles in the margin.

Once we have our FacetGrid, we call the **map** function in order to map data onto our grid. First, we tell it what type of plot we want to use. In this example, we use a line plot from `Matplotlib`. We then pass the variable we want plotted, **passengers**.

This creates a line plot for each row for our variable of interest.

PairGrid

Another common multi-plot function in Seaborn is `PairGrid()`. This function plots pairwise relationships in a grid. You generally pass one parameter which is a dataframe consisting of the columns you wish to plot.

You then call the `map()` function which maps the pairwise combinations of your columns onto the plot of your choice. The plotting function you wish to use is the parameter you pass to `map()`.

Here is an example:

```
from sklearn.datasets import load_boston
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# Set the palette and style to be more minimal
sns.set(style='ticks', palette='Set2')

# Load data as explained in introductory lesson
boston_data = load_boston()
boston_df = pd.DataFrame(boston_data.data, columns=boston_data.feature_names)

# Create and map the PairGrid
g = sns.PairGrid(boston_df[['CRIM', 'NOX', 'INDUS']])
g.map(plt.scatter);

# Remove excess chart lines and ticks for a nicer looking plot
sns.despine()
```

The result, as you see, is all of the pairwise scatterplots of your variables allowing you to quickly scan for potential relationships. Since the diagonal of your grid is the scatter plot of variables with themselves, you will always get data on the 45-degree line.

You have a lot of ways you can experiment with the FacetMap and map combination, so have fun! You can learn more from the Seaborn documentation.

Now that you are familiar with techniques to visualize any dataset, the next lesson brings some challenges for you to solve.

Exercise: Visualizing Auto MPG Dataset

This lesson provides a few exercises to test your understanding of the seaborn module for visualization.

WE'LL COVER THE FOLLOWING

- Scatter plot
- Bar plot
- Line plot

In the challenges below, we will use the same data frame as read in from the Reading Data exercise.

Scatter plot

Create an additional function `scatter_plot` that takes a data frame `df` and plot relationship between the following two attributes:

- `displacement` over **x-axis**
- `acceleration` over **y-axis**

Try to implement the function below. Good Luck!

```
import pandas as pd
import seaborn as sns

# Load data
def read_csv():
    # Define the column names as a list
    names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "moc
    # Read in the CSV file from the webpage using the defined column names
    df = pd.read_csv("auto-mpg.data", header=None, names=names, delim_whitespace=True)
    return df

# Create the scatter plot
def scatter_plot(df):
    return

# Call the function
scatter_plot(read_csv())
```

Bar plot

Create an additional function `bar_plot` that takes a data frame `df` and compares the `cylinders` of all the cars from **1975** `model_year` and `ford` company.

To check whether a car is owned by `ford` company, you can use the following line:

```
df.car_name.str.contains('ford')
```

This statement will only be *true* for the cars of the `ford` company, that's how you can filter the cars.

To check whether a car belongs to `model-year` of **1975**, you can use the following line:

```
df["model_year"].isin([75])
```

This statement will only be *true* for the cars of the **1975** `model_year`.

Try to implement the function below. Good Luck!

```
import pandas as pd
import seaborn as sns

# Load data
def read_csv():
    # Define the column names as a list
    names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "moc
    # Read in the CSV file from the webpage using the defined column names
    df = pd.read_csv("auto-mpg.data", header=None, names=names, delim_whitespace=True)
    return df


def bar_plot(df):
    return

# Call the function
bar_plot(read_csv())
```

Line plot

Create an additional function `line_plot` that takes a data frame `df` and then plots the change in `weight` throughout the years for the cars of `ford` company.

Try to implement the function below. Good Luck!

```
import seaborn as sns            # importing seaborn functionality
import pandas as pd


# Load dataset
def read_csv():
    # Define the column names as a list
    names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "moc
    # Read in the CSV file from the webpage using the defined column names
    df = pd.read_csv("auto-mpg.data", header=None, names=names, delim_whitespace=True)
    return df


def line_plot(df):
    return

# Call the function
line_plot(read_csv())
```

We hope that you were able to solve the challenges. The next lesson brings you the solutions to the above challenges.

Solution Review: Visualizing Auto MPG Dataset

This lesson provides the solutions to the previous challenges.

WE'LL COVER THE FOLLOWING

- Scatter plot
- Bar plot
- Line plot

Scatter plot

```
import pandas as pd
import seaborn as sns

# Load data
def read_csv():
    # Define the column names as a list
    names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "moc
    # Read in the CSV file from the webpage using the defined column names
    df = pd.read_csv("auto-mpg.data", header=None, names=names, delim_whitespace=True)
    return df

# Create the scatter plot
def scatter_plot(df):
    sns.lmplot(x="displacement", y="acceleration", data = df)
    # Remove excess chart lines and ticks for a nicer looking plot
    sns.despine()

# calling function
scatter_plot(read_csv())
```

According to the problem statement, we need to find the relationship between `acceleration` and `displacement`. To plot a visualization, we import `seaborn` module at **line 2**. Before doing it, we have to read the data first. There's no need to explain how to read the data, as we studied that in detail previously. Dataset is read from **line 5** to **line 10**.

Moving towards the main implementation, look at the header of the `scatter_plot(df)` function at **line 13**. It takes *one* arguments as input:

- `df`: A dataframe containing the dataset in the form of a matrix.

Line 14 is the most important line. We are using a built-in function `lmplot` from `seaborn` library which takes *three* main argument:

- `x`: Column whose values are to plotted at the x-axis
- `y`: Column whose values are to plotted at the y-axis
- `data`: Dataframe containing data in the form of a matrix

We set `displacement` as `x` and `acceleration` as `y` according to the problem statement. At **line 16**, we are just trying to make the plot look clean using function `despine()`.

At **line 19**, we are calling the function `scatter_plot(read_csv())`. First control will transfer to `read_csv()` at **line 5** and we'll get a dataframe. Then control will go to **line 13** and plot will be visualized.

The plot clearly shows the negative correlation between these two data values, which means `acceleration` is the inverse of `displacement`.

Bar plot

```
import pandas as pd
import seaborn as sns

# Load data
def read_csv():
    # Define the column names as a list
    names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "moc
    # Read in the CSV file from the webpage using the defined column names
    df = pd.read_csv("auto-mpg.data", header=None, names=names, delim_whitespace=True)
    return df


def bar_plot(df):
    df = df[df.car_name.str.contains('ford')]
    df = df[df["model_year"].isin([75])]

    # Create the bar plot
    sns.barplot(df['car_name'], df['cylinders'])
    # Remove excess chart lines and ticks for a nicer looking plot
    sns.despine()
```

```
bar_plot(read_csv())
```

According to the problem statement, we need to compare the `cylinders` of all the cars from **1975** `model_year` and `ford` company. To plot a visualization, we import `seaborn` module at **line 2**. Before doing it we definitely have to read the data first. There's no need to explain how to read the data, as we studied that in detail previously. Dataset is read from **line 5** to **line 10**.

Moving towards the main implementation, look at the header of the `bar_plot(df)` function at **line 13**. It takes *one* arguments as input:

- `df`: A dataframe containing the dataset in the form of a matrix.

Before plotting, we need to filter out the data we need. Look at **line 14**. Here we are considering all the rows for which the value in `car_name` would have `ford` as a substring. For example, **chevrolet chevelle** will be discarded and **ford torino** will be considered as a data value. Next, we are keeping all the rows for which `model_year` is **1975**. So all the instances having `model_year` other than **1975** will be discarded.

Line 18 is the most important line. We are using a built-in function `barplot` from `seaborn` library which takes *two* main argument:

- **column1**: Column whose values are to plotted at the x-axis
- **column2**: Column whose values are to plotted at the y-axis

We set `car_name` as `x` and `cylinders` as `y` according to the problem statement. At **line 20**, we are just trying to make the plot look clean using function `despine()`.

At **line 22**, we are calling the function `bar_plot(read_csv())`. First control will transfer to `read_csv()` at **line 5** and we'll get a dataframe. Then control will go to **line 13** and plot will be visualized.

The plot clearly shows that the `ford` company launches following *four* different models in **1975**:

- `ford maverick` having **6** cylinders

- `ford ltd` having **8** cylinders
- `ford mustang ii` having **8** cylinders
- `ford pinto` having **5** cylinders

Line plot

```
import seaborn as sns            # importing seaborn functionality
import pandas as pd

# Load dataset
def read_csv():
    # Define the column names as a list
    names = ["mpg", "cylinders", "displacement", "horsepower", "weight", "acceleration", "moc
    # Read in the CSV file from the webpage using the defined column names
    df = pd.read_csv("auto-mpg.data", header=None, names=names, delim_whitespace=True)
    return df

def line_plot(df):

    # filtering the dataset to obtain the January records for all years
    df=df[df.car_name.str.contains('ford')]

    #plotting a line graph
    sns.lineplot(df.model_year, df.weight)

line_plot(read_csv())
```

According to the problem statement, we need to plot the change in the `weight` of all the `ford` cars. To plot a visualization, we import the `seaborn` module at **line 2**. Before doing it, we have to read the data first. There's no need to explain how to read the data, as we studied that in detail previously. Dataset is read from **line 5** to **line 10**.

Moving towards the main implementation, look at the header of the `line_plot(df)` function at **line 12**. It takes *one* arguments as input:

- `df`: A dataframe containing the dataset in the form of a matrix.

Before plotting, we need to filter out the data we need. Look at **line 15**. Here we are considering all the rows for which the value in `car_name` would have `ford` as a substring. For example, **chevrolet chevelle** will be discarded, and **ford torino** will be considered as a data value.

Line 18 is the most important line. We are using a built-in function

`lineplot()` from `seaborn` library which takes *two* main argument:

- **column1**: Column whose values are to plotted at the x-axis
- **column2**: Column whose values are to plotted at the y-axis

We set `model_year` as `x` and `weight` as `y` according to the problem statement.

At **line 20**, we are calling the function `line_plot(read_csv())`. First control will transfer to `read_csv()` at **line 5** and we'll get a dataframe. Then control will go to **line 12** and plot will be visualized.

The plot marks the points, each for one `model_year`. The point represents the average `weight` of all the cars from the `ford` company in a year. Then those points are connected via a *line* to form a plot.

That's all about the solutions. In the next lesson, there's a quick quiz for you to evaluate your concepts regarding visualization. Good Luck!

Test Yourself

A quiz to test your knowledge of visualization and the appropriate functions

WE'LL COVER THE FOLLOWING

- Quiz

Quiz

Take the quiz below to test your knowledge of visualization and `Seaborn` functions

Visualization Quiz

1 Which `Seaborn` function creates *scatter* plots?

COMPLETED 0%

1 of 4

That’s it for our **Exploratory Data Analysis and Visualization for Python** course. I hope you learned a lot about how to analyze and visualize your data in Python!